

Predictive Maintenance for Wind Turbine Failure Detection Using Neural Networks

ReneWind Predictive Maintenance Project
Neural Networks and Deep Learning

May 25, 2026

Contents / Agenda

- Executive Summary
- Business Problem Overview and Solution Approach
- Data Overview
- EDA Results
- Data Preprocessing
- Model Evaluation Criterion
- Model Performance Summary
- Test Performance of Best Model
- Actionable Insights & Recommendations
- Conclusion
- Appendix

Executive Summary

Developed a neural network-based predictive maintenance model to identify wind turbine generator failures using turbine sensor data.

Performed extensive exploratory data analysis (EDA), which revealed significant outliers, varying sensor behavior, and a highly imbalanced target variable with only ~5.5% failure events.

Multiple neural network architectures were evaluated using dropout regularization, class weighting, and different network configurations.

Model 3 was selected as the best-performing model based on its balance between Recall, Precision, F1 Score, and overall model stability.

The model successfully identified approximately 87–93% of turbine failure events while maintaining very high overall accuracy.

The final model demonstrated strong generalization across training, validation, and testing datasets, indicating reliable performance on unseen data.

Actionable Insights

- Sensor abnormalities and extreme operating conditions contain strong predictive signals for turbine failure detection.
- Handling class imbalance significantly improved the model's ability to detect rare failure events.
- Models using regularization and class weighting achieved stronger operational performance compared to simpler baseline models.
- Early identification of abnormal turbine behavior can support proactive maintenance planning and reduce unexpected failures.

Business Recommendations

- Deploy Model 3 as a predictive maintenance support tool for turbine monitoring systems.
- Use model predictions to prioritize maintenance inspections for high-risk turbines.
- Continuously retrain the model with new turbine operational data to maintain performance over time.
- Integrate the predictive model with existing maintenance scheduling systems to reduce downtime and optimize maintenance operations.
- Conduct pilot testing before large-scale deployment to validate performance under real-world operating conditions.

Business Impact

- The predictive maintenance solution can help organizations reduce emergency repair costs, minimize unplanned downtime, improve turbine reliability, optimize maintenance scheduling, and increase operational efficiency.
- Early failure detection also supports safer turbine operations, improved energy production consistency, and more data-driven maintenance decision-making.

Business Problem Overview and Solution Approach

Business Context

- Renewable energy plays a critical role in improving sustainability and reducing environmental impact. Wind turbines are one of the most widely adopted renewable energy technologies worldwide.
- ReneWind aims to improve turbine operational efficiency through predictive maintenance using machine learning and sensor-based monitoring systems.
- Wind turbines continuously collect operational and environmental sensor data such as temperature, humidity, wind speed, vibration, and component-level measurements.
- Unexpected generator failures can result in significant operational downtime, expensive replacement costs, lost energy production, and increased maintenance expenses.
- Predicting failures before breakdown occurs allows organizations to proactively repair turbines and improve overall reliability.

Project Objective

- The objective of this project is to develop neural network classification models capable of predicting generator failures using turbine sensor data.

Dataset Overview

- 20,000 observations in the training dataset
- 5,000 observations in the testing dataset
- 40 numerical predictor variables
- 1 binary target variable

Business Problem Overview and Solution Approach

Target Variable

- Target = 1 → Failure
- Target = 0 → No Failure
- The goal is to identify turbine failures early so maintenance teams can intervene before complete generator breakdown occurs.

Business Impact of Prediction Outcomes

1. True Positive (TP)

- Correctly predicting a turbine failure.
- Enables proactive maintenance
- Reduces risk of catastrophic turbine breakdown
- Preferred outcome because repair costs are lower than replacement costs

2. False Negative (FN)

- Failure occurs but is not detected by the model
- Business Impact:
- Most critical and expensive error
- Can lead to severe turbine damage, emergency shutdowns, replacement costs, and operational downtime
- Because missed failures are highly costly, minimizing False Negatives is the primary business objective.

3. False Positive (FP)

- Model predicts failure when no failure occurs.

Business Impact:

- Leads to additional inspections or maintenance checks
- Increases operational costs slightly
- Less costly than unexpected turbine breakdowns

Key Modeling Priority

Because turbine failures are rare but operationally expensive, Recall was prioritized as the primary evaluation metric to maximize failure detection capability.

Business Problem Overview and Solution Approach

Project Methodology

The following approach was used to solve the business problem:

1. Exploratory Data Analysis (EDA)

- Analyzed sensor distributions
- Identified outliers and skewness
- Evaluated class imbalance
- Investigated feature relationships and correlations

2. Data Preprocessing

- Checked for missing values and duplicates
- Applied median imputation for missing data
- Split data into training, validation, and testing datasets
- Prepared data for neural network modeling

3. Neural Network Model Development

- Multiple neural network architectures were developed and evaluated using different hidden layer configurations, dropout regularization, optimizer tuning, and class weighting techniques.

4. Model Performance Improvement Techniques

- The following techniques were used to improve predictive performance:
 - Additional hidden layers
 - Dropout regularization to reduce overfitting
 - Handling class imbalance using class weights
 - Architecture tuning
 - Optimizer comparison (SGD vs Adam)

5. Model Evaluation Metrics

Models were evaluated using:

- Recall
- Precision
- F1-Score
- Accuracy
- Classification Reports
- Confusion Matrix Analysis

Special emphasis was placed on Recall because detecting failures early is the key business objective.

Business Problem Overview and Solution Approach

Final Model Selection

- Model 3 was selected as the final model because it achieved the best balance between Recall, Precision, F1-Score, and overall model stability.
- The model also demonstrated strong generalization across training, validation, and testing datasets.

Final Test Performance (Model 3)

Failure Detection Performance (Class 1)

- Precision: 90%
- Recall: 87%
- F1-Score: 88%

Business Outcome

The final neural network model can help ReneWind detect turbine failures before breakdown occurs, reduce emergency maintenance costs, minimize operational downtime, improve maintenance scheduling efficiency, and increase turbine reliability.

Final Recommendation

- Deploy Model 3 as a predictive maintenance support system for turbine monitoring operations.
- Continuously retrain the model with new operational data to maintain performance over time.
- Integrate model predictions with maintenance planning systems for proactive decision-making.

Data Overview and Initial Findings: Missing Value Analysis

Missing Value Analysis

- The missing value analysis shows that:
- Only two variables, **V1** and **V2**, contain missing values.
- Both variables have **18 missing observations** each.
- All remaining predictor variables and the target variable contain no missing values.
- Since the dataset contains 20,000 observations, the percentage of missing values is extremely small.

Importance of Missing Value Analysis

- Checking for missing values is important because:
- Missing values can negatively impact model performance and may cause errors during training.
- Many machine learning algorithms cannot process null values directly.
- Proper treatment of missing values improves model reliability and prediction accuracy.
- Since the number of missing values is very low, simple imputation techniques such as mean or median imputation can be applied effectively without significantly changing the data distribution.

```
[21]: # let's check for missing values in the test data
data_test.isnull().sum() #
```

```
[21]: V1      5
      V2      6
      V3      0
      V4      0
      V5      0
      V6      0
      V7      0
      V8      0
      V9      0
      V10     0
      V11     0
      V12     0
      V13     0
      V14     0
      V15     0
      V16     0
      V17     0
      V18     0
      V19     0
      V20     0
      V21     0
      V22     0
      V23     0
      V24     0
      V25     0
      V26     0
      V27     0
      V28     0
      V29     0
      V30     0
      V31     0
      V32     0
      V33     0
      V34     0
      V35     0
      V36     0
      V37     0
      V38     0
      V39     0
      V40     0
      Target  0
      dtype: int64
```


Data Overview and Initial Findings: Statistical Summary

Statistical Summary Observations

- The statistical summary provides important insights into the numerical variables in the dataset.
- Key observations include:
- The dataset contains **40 predictor variables** and **1 target variable**.
- Several variables have relatively large standard deviations, indicating high variability in sensor readings.
- The wide gap between minimum and maximum values suggests the possible presence of outliers in multiple features.
- The target variable has a mean value of approximately **0.0555**, indicating that only about **5.5%** of observations correspond to generator failures.

Importance of Statistical Summary

- The statistical summary is important because:
- It helps understand the distribution and spread of the variables.
- It helps identify potential outliers that may affect model performance.
- It provides insight into whether scaling or normalization may be required before modeling.
- The low proportion of failure cases indicates that the dataset is **highly imbalanced**.

Data Overview and Initial Findings

Business and Modeling Implications

- Class imbalance is critical in this project because:
- A model trained on imbalanced data may become biased toward predicting the majority class (“No Failure”).
- This can result in poor detection of actual failures.
- False Negatives are especially costly because undetected failures can lead to expensive generator replacement and operational downtime.
- Therefore, the final model should prioritize identifying failures effectively by improving Recall and reducing False Negatives, even if it results in slightly higher inspection costs from False Positives.
- Because of this imbalance, evaluation metrics such as **Recall, Precision, F1-score, and Confusion Matrix** are more meaningful than accuracy alone.

EDA Results: Univariate Analysis and Business Interpretation

Overview

Univariate analysis was conducted on all numerical variables (V1–V40) and the target variable to better understand:

- the distribution of each feature, variability in sensor behavior, the presence of outliers, skewness, and potential preprocessing requirements before machine learning modeling.
- Histograms and boxplots were used to evaluate the distribution and spread of each variable individually.

Dataset Characteristics

- The dataset contains:

Training Dataset

- 20,000 observations
- 40 numerical predictor variables
- 1 binary target variable

Testing Dataset

- 5,000 observations
- 40 numerical predictor variables
- 1 binary target variable

The target variable is:

- **0 = No Failure**
- **1 = Failure**

Missing Value Analysis

Observations

- Only variables **V1** and **V2** contain missing values.
- Both variables contain only **18 missing observations** each.
- All remaining variables and the target variable contain no missing values.
- The percentage of missing values is extremely small relative to the overall dataset size.

Why Missing Value Analysis Is Important

Checking for missing values is important because:

- Many machine learning algorithms cannot process null values directly.
- Missing data can negatively impact prediction accuracy and model stability.
- Improper handling of missing values may introduce bias into the model.
- Since missingness is minimal, simple imputation methods such as: mean imputation, median imputation, or SimpleImputer can effectively handle the issue without significantly altering the data distribution.

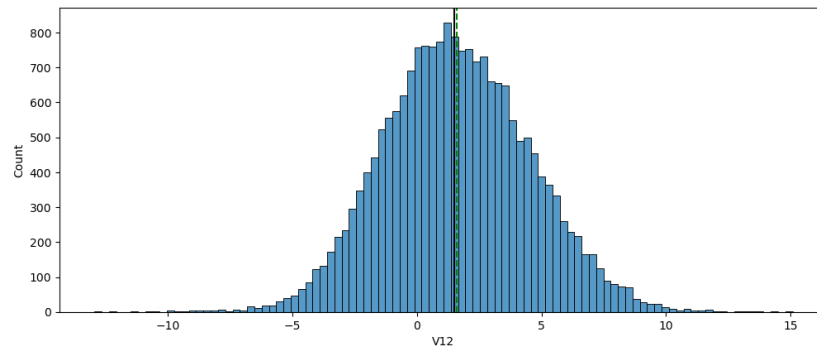
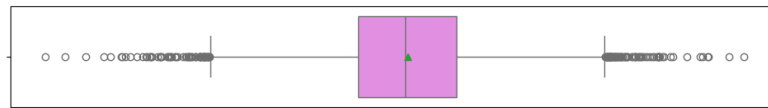
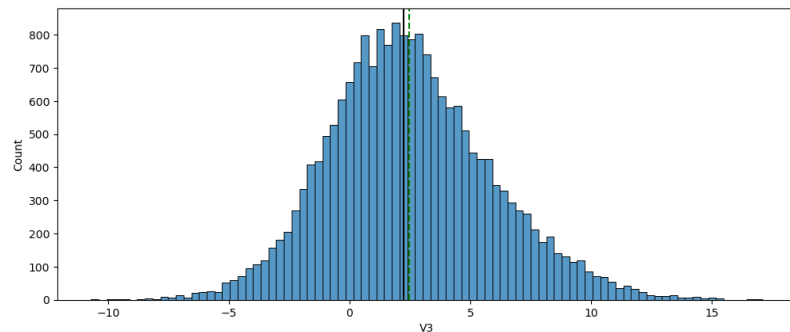
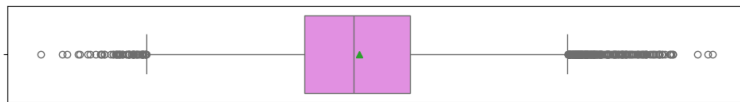
Business Impact of Missing Values

- Proper treatment of missing sensor readings helps improve prediction reliability, reduce operational risk, and support more accurate maintenance decisions.
- Ignoring missing values could increase the likelihood of inaccurate predictions and unexpected turbine failures.

EDA Results: Univariate Analysis –Variables V1 to V21

Observations

- Most variables from **V1 to V21** follow approximately bell-shaped, normal, or near-normal distributions.
- Several variables display slight positive skewness, slight negative skewness, and moderate asymmetry.



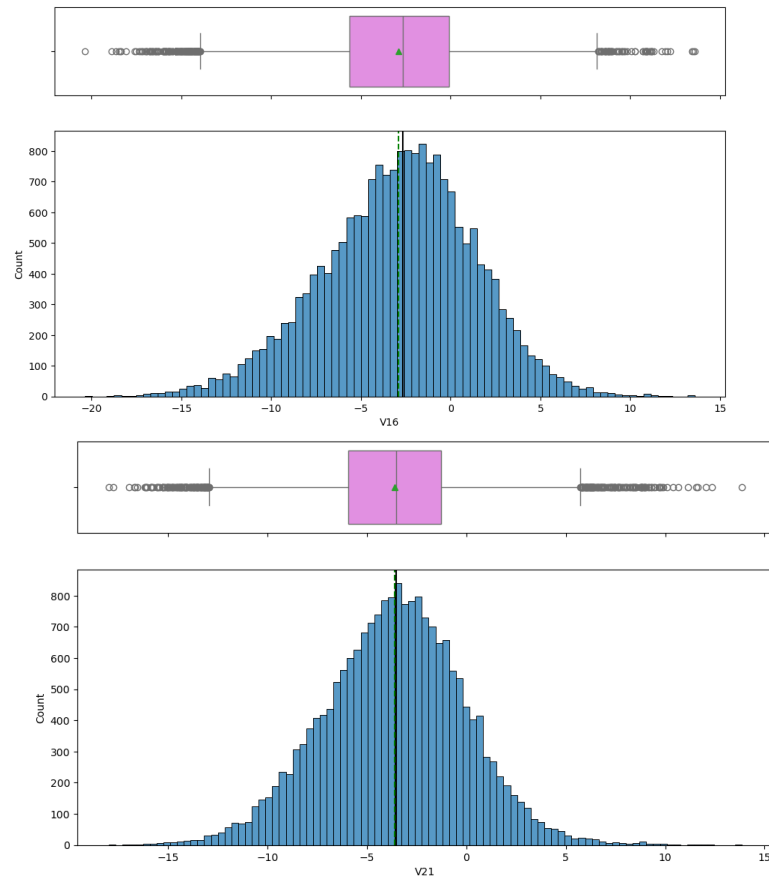
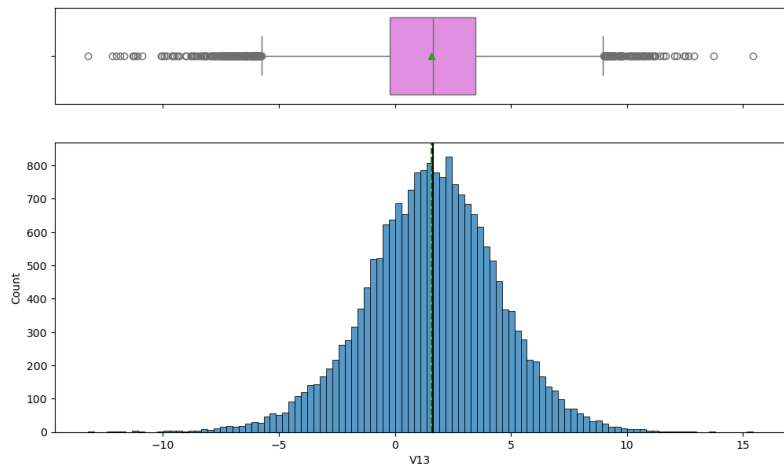
[Link to Appendix slide on data background check](#)

EDA Results: Univariate Analysis –Variables V1 to V21

Many variables contain visible outliers on both lower and upper ends of the distributions. The mean and median are relatively close for many variables, indicating moderate symmetry and stable sensor behavior. Variables such as **V3**, **V12**, **V13**, **V16**, **V21** show wider spreads, heavier tails, and higher variability.

Histograms indicate that most variables are continuous numerical features with varying scales and ranges.

Outliers are common across many variables, which is expected in real-world industrial sensor data.



EDA Results: Univariate Analysis –Variables V1 to V21

Why This Is Important for EDA

Performing univariate analysis is important because it helps understand the behavior of each feature individually, identify skewness and unusual patterns, detect extreme values and operational anomalies, and determine whether preprocessing is required.

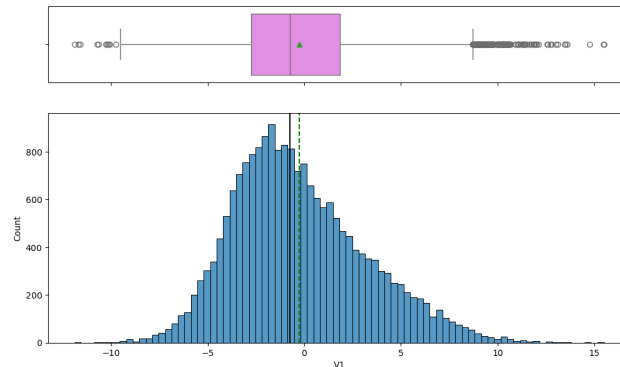
This analysis helps determine whether scaling, normalization, transformation, or robust preprocessing techniques may improve machine learning performance.

Since neural networks are sensitive to feature scale and distribution, understanding variable behavior is critical before model development.

Business Interpretation – Variables V1 to V21

From a business perspective:

- Extreme sensor readings may indicate abnormal turbine behavior, overheating, vibration abnormalities, mechanical stress, or early signs of equipment degradation.
- Variables with high variability and frequent outliers may contain strong predictive signals for turbine failure.
- Detecting abnormal operating conditions early can help reduce downtime lower replacement costs, improve turbine reliability, and optimize maintenance planning.



EDA Results: Univariate Analysis –Variables V20 to V39

Observations

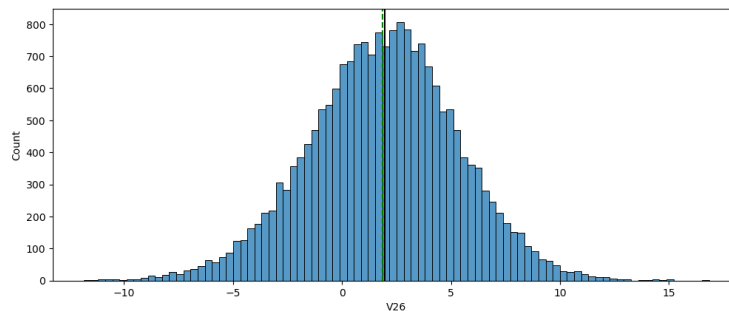
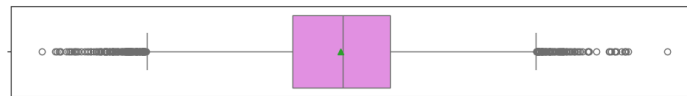
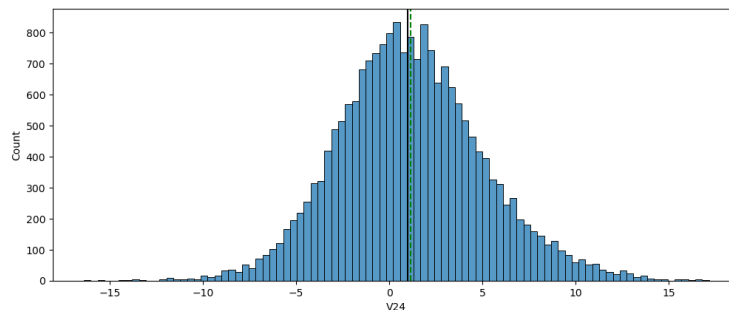
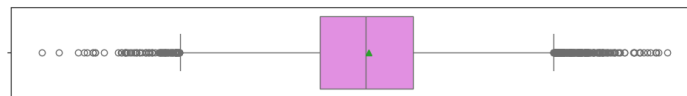
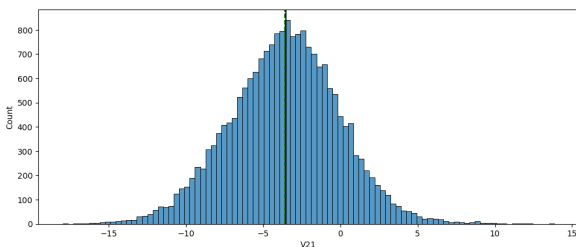
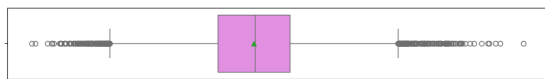
Variables from **V20 to V39** also display distributions that are mostly: approximately normal, bell-shaped, or near-symmetric.

Several variables exhibit moderate skewness, particularly **V21, V24, V26, V27, V31, V32, V36**.

These variables show longer tails, asymmetric distributions, and larger variability.

Additional observations include:

- Many variables contain substantial outliers on both sides of the distribution.
- Variables such as **V32** and **V36** display especially wide ranges and heavier tails.
- Some variables are tightly concentrated around the center, while others are more dispersed.
- Mean and median lines remain relatively close for several variables despite the presence of outliers.
- Mild positive skewness is observed in some variables where right tails extend further than left tails.



EDA Results: Univariate Analysis –Variables V20 to V39

Why This Is Important for EDA

Analyzing these variables is important because it helps identify whether the data satisfies assumptions commonly used in machine learning and statistical modeling, highly skewed variables may require preprocessing, extreme outliers may strongly influence model performance, and wider distributions may indicate abnormal operational conditions.

Variables with heavy tails and large spreads may contain useful information related to turbine instability, mechanical stress, and generator failure behavior.

This analysis also helps determine whether feature engineering, dimensionality reduction, or advanced preprocessing may improve model performance later in the project.

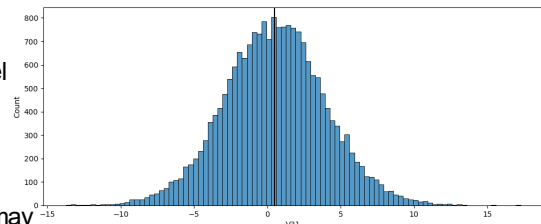
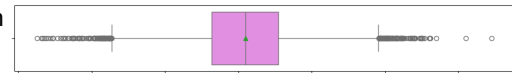
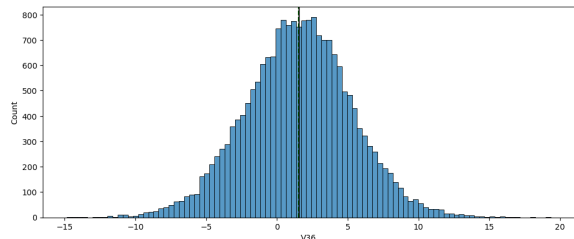
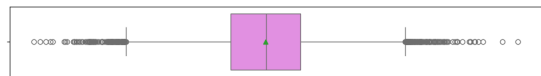
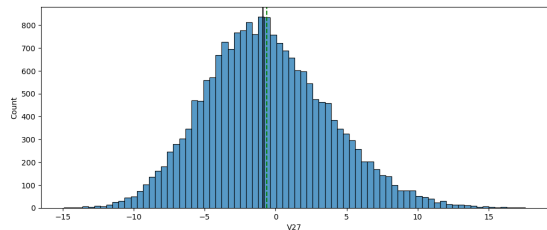
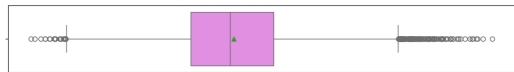
Business Interpretation – Variables V20 to V39

From a business and operational perspective extreme sensor readings may represent overheating, vibration irregularities, unstable operating conditions, or early component degradation.

Variables with larger variability and extreme outliers may be particularly useful for predicting turbine failure events.

Detecting unusual sensor behavior early can help reduce costly unplanned downtime, improve maintenance scheduling, and support predictive maintenance strategies.

Because turbine failures are rare but expensive, identifying subtle abnormal patterns in these variables may significantly improve maintenance decision-making.



EDA Results: Univariate Analysis –Variables V40

Observations

Variable **V40** follows an approximately bell-shaped distribution similar to many earlier sensor variables.

Additional observations include:

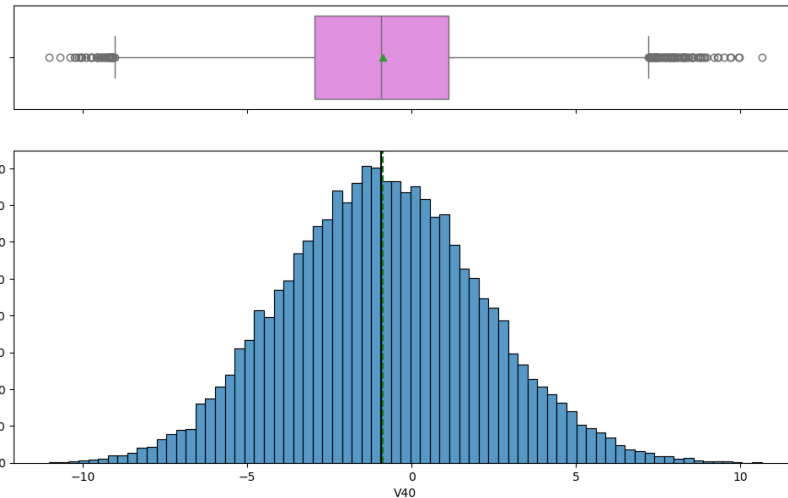
- The histogram appears relatively symmetric with mild skewness.
- The mean and median are very close, indicating balanced behavior.
- Several outliers are visible on both lower and upper tails.

Compared to earlier variables, V40 appears slightly more stable with a narrower spread.

Importance of Variable Distribution Analysis

Analyzing variable distributions helps identify skewness, variability, outliers, and potential data quality concerns.

These insights help determine whether scaling is required, whether transformations may improve model performance, and which variables may contribute strongly to predictive accuracy.



EDA Results: Univariate Analysis --Distribution of Target Variable

Business Impact of Sensor Distribution Analysis

Stable sensor behavior improves the model's ability to distinguish between healthy turbine operation, and abnormal operating conditions.

Variables showing unusual or extreme behavior may act as early warning indicators for turbine failure and predictive maintenance systems.

Target Variable Analysis

Observations

The target variable shows a highly imbalanced distribution.

- Most observations belong to class **0 (No Failure)**.
- Only a small proportion belong to class **1 (Failure)**.

This confirms:

- approximately **94.5% No Failure**
- approximately **5.5% Failure**

The boxplot also highlights the strong imbalance between the two classes.

Why Target Distribution Is Important

1. Class Imbalance Impacts Model Training

Machine learning models may become biased toward the majority class.

For example:

A model predicting only "No Failure" could still achieve high accuracy.

However, it would fail to identify actual turbine failures.

2. Accuracy Alone Becomes Misleading

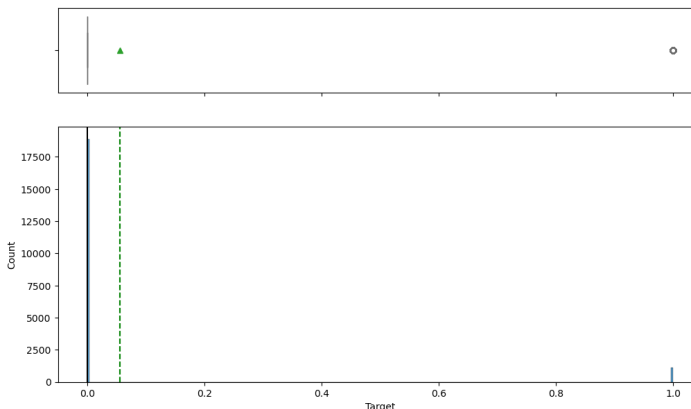
Because failures are rare accuracy alone is not a reliable evaluation metric. More meaningful metrics include Recall, Precision, F1-score, ROC-AUC, and confusion matrix analysis.

3. Failure Detection Is the Main Business Objective

The business problem prioritizes early failure detection.

Missing a true failure (False Negative) may result in turbine breakdown, expensive generator replacement, operational downtime, and increased maintenance expenses.

Therefore, the final model should prioritize maximizing Recall, minimizing False Negatives, and improving failure detection performance.



EDA Results: Univariate Analysis -- Distribution of Target Variable

Observations

- The distribution of the target variable in both the training and testing datasets is highly imbalanced.

Training Dataset

- Class 0 (No Failure): 94.45%
- Class 1 (Failure): 5.55%

Testing Dataset

- Class 0 (No Failure): 94.36%
- Class 1 (Failure): 5.64%

The proportions in the training and testing datasets are very similar, indicating that both datasets follow a consistent distribution.

This means turbine failures are relatively rare events, while the majority of turbines operate under normal conditions.

Why This Is Important

Analyzing the distribution of the target variable is extremely important because:

1. The Dataset Is Highly Imbalanced

- The model will be trained on a dataset where the majority class ("No Failure") dominates the observations.
- Without proper handling:
- the model may become biased toward predicting the majority class,
- resulting in poor failure detection performance.
- For example a model predicting only "No Failure" could still achieve over 94% accuracy, but it would completely fail to identify actual turbine failures.

2. Accuracy Alone Becomes Misleading

Because failure events are rare accuracy is not an appropriate standalone evaluation metric.

Again, this shows that a more meaningful evaluation metrics include Recall, Precision, F1-score, ROC-AUC, and confusion matrix analysis.

These metrics provide a better understanding of how effectively the model identifies failures.

3. Class Imbalance Affects Model Optimization

The imbalance suggests that additional techniques may be needed during model development, such as class weighting, SMOTE (Synthetic Minority Oversampling), threshold tuning, or specialized evaluation strategies. These methods help improve the model's ability to detect rare failure events.

Checking the distribution of

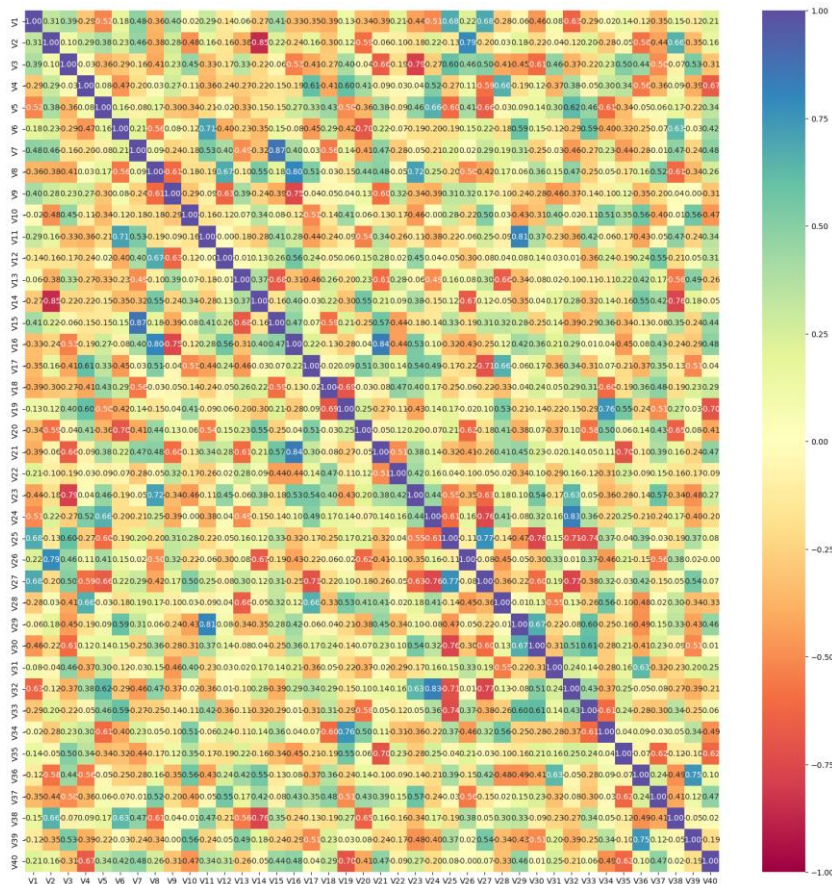
```
# For train data
df["Target"].value_counts()

Target
0    0.9445
1    0.0555
Name: proportion, dtype: float64

# For test data
df_test["Target"].value_counts()

Target
0    0.9436
1    0.0564
Name: proportion, dtype: float64
```

EDA Results: Bivariate Analysis – Correlation Heatmap



Observations from the Correlation Matrix

The heatmap displays the pairwise correlation coefficients between all numerical variables (V1–V40).

Correlation values range from:

- **+1** → strong positive relationship
- **-1** → strong negative relationship
- **0** → little or no linear relationship

Key Observations

Most variables show weak to moderate correlations, indicating that many sensor variables capture different operational characteristics of the turbines.

Several variable pairs exhibit strong positive correlations, where variables increase together.

Examples include V1 with V25 and V27, V11 with V29, V24 with V32, V30 with V33

Some variable pairs show strong negative correlations, meaning as one variable increases, the other decreases.

Examples include: V2 with V14, V18 with V35, V25 with V31

Not extremely perfect correlations are observed between most variables, which is beneficial because the dataset does not appear heavily dominated by duplicate information. Certain clusters of variables display similar correlation patterns, suggesting that some sensors may be monitoring related turbine behaviors or interconnected mechanical systems.

EDA Results: Bivariate Analysis – Correlation Heatmap

Why Correlation Analysis Is Important in EDA

Correlation analysis is one of the most important parts of Exploratory Data Analysis because it helps identify relationships between variables.

1. Detecting Multicollinearity

Highly correlated variables may contain redundant information.

This is important because multicollinearity can negatively affect certain machine learning models, especially linear models such as Logistic Regression.

Strong multicollinearity may reduce model interpretability, increase model instability, and inflate coefficient importance.

2. Understanding Relationships Between Sensors

Correlation analysis helps identify which turbine sensors move together, which variables behave independently, and which measurements may represent similar operational conditions.

This provides insight into turbine mechanics, operational dependencies, and system interactions.

3. Supporting Feature Selection

Highly correlated features may provide overlapping information, or contribute limited additional predictive value.

This analysis helps determine whether feature reduction, dimensionality reduction, or feature engineering may improve model performance.

4. Improving Machine Learning Performance

Understanding variable relationships helps reduce noise, improve model stability, and optimize preprocessing decisions.

For example, strongly correlated features may benefit tree-based models, while redundant variables may hurt simpler linear models.

EDA Results: Bivariate Analysis – Correlation Heatmap

Business Interpretation

From a business and operational perspective correlated variables may represent connected turbine subsystems such as temperature, pressure, vibration, rotational speed, or generator load.

Strong positive correlations may indicate components that react together during normal operation, or shared operational behavior across turbine systems.

Strong negative correlations may indicate balancing mechanisms, inverse operational relationships, or compensating system responses.

Changes in correlated sensor groups may help identify abnormal turbine conditions, mechanical degradation, or early failure patterns.

Business Impact

Understanding correlations between turbine sensors can significantly improve predictive maintenance strategies.

Operational Benefits

A better understanding of sensor relationships can help detect abnormal operating conditions earlier, improve failure prediction accuracy, reduce unnecessary maintenance inspections, and optimize monitoring systems.

Maintenance Optimization

Correlation analysis may help engineers identify which sensors provide the most useful operational information, reduce redundant monitoring, and focus attention on high-risk turbine components.

Cost Reduction

Improved predictive modeling can help reduce generator replacement costs, unplanned downtime, operational interruptions, and maintenance expenses.

Overall Conclusion

The correlation heatmap suggests that the dataset contains meaningful relationships between turbine sensor variables, moderate-to-strong correlations exist between several features, and the data appears suitable for machine learning modeling.

The absence of excessive perfect correlations is beneficial because the model is less likely to suffer from severe redundancy issues, while still retaining useful inter-variable relationships that may improve turbine failure prediction.

Data Preprocessing: Data Preparation for Modeling

Observations

The dataset was successfully divided into training, validation, and testing datasets for machine learning model development.

Training Dataset

Shape: (16000, 40)

Contains 16,000 observations and 40 predictor variables

Validation Dataset

Shape: (4000, 40)

Contains 4,000 observations and 40 predictor variables

Testing Dataset

Shape: (5000, 40)

Contains: 5,000 observations and 40 predictor variables

The target variable (Target) was successfully separated from the predictor variables before model training.

Data Preparation for Modeling

```
1: # Dividing train data into X and y
X = data.drop(columns = ["Target"], axis=1) # Complete the code to remove the column named 'Target'
y = data["Target"] # Complete the code to select the column named 'Target'
```

Since we already have a separate test set, we don't need to divide data into train, validation and test

```
1: # Splitting data into training and validation set:

X_train, X_val, y_train, y_val = train_test_split(
    X, y, test_size=0.2, random_state=1, stratify=y # Complete the code to define the test size
)

1: # Checking the number of rows and columns in the X_train data
X_train.shape

1: (16000, 40)

1: # Checking the number of rows and columns in the X_val data
X_val.shape

1: (4000, 40)

1: # Dividing test data into X_test and y_test
X_test = data_test.drop(columns = ["Target"], axis=1) # Complete the code to remove the target column
y_test = data_test["Target"] # Complete the code to select the target column

1: # Checking the number of rows and columns in the X_test data
X_test.shape

1: (5000, 40)
```


Data Preprocessing: Data Preparation for Modeling

Why This Is Important

Preparing the data correctly before modeling is one of the most critical steps in machine learning.

1. Separation of Features and Target Variable

The predictor variables (X) and target variable (y) were separated to allow the model to learn relationships between sensor readings and turbine failures, avoid including the target variable as an input feature, and ensure proper supervised learning. This is necessary because the target variable represents the output the model is trying to predict.

2. Splitting the Dataset

The training dataset was split into training data (80%) and validation data (20%). This is important because with the **Training Data**, the model learns patterns from the training dataset. With the **Validation Data** the validation dataset is used to evaluate model performance during training, tune hyperparameters, compare models, and prevent overfitting. The separate testing dataset is reserved for final unbiased model evaluation.

3. Stratified Sampling

The parameter **stratify=y** ensures that the class distribution remains consistent across training, validation, and testing datasets.

This is especially important because the target variable is highly imbalanced.

Without stratification one dataset might contain too few failure cases, leading to unreliable model evaluation and poor generalization.

Maintaining similar class distributions across datasets helps ensure that the model learns realistic patterns, evaluation metrics remain reliable, and model performance reflects real-world operating conditions.

4. Preventing Data Leakage

The testing dataset was kept completely separate from the training and validation process. This is important because the model should never learn from unseen test data during training, and the testing dataset should only be used for final evaluation.

Preventing data leakage ensures unbiased model evaluation, realistic performance measurement, and better generalization to future turbine data.

Data Preprocessing: Data Preparation for Modeling

Business Impact

Proper data preparation directly impacts the reliability and effectiveness of the predictive maintenance system.

1. Improved Failure Detection

Well-structured training and validation datasets help the model learn meaningful turbine behavior patterns, identify abnormal operating conditions, and improve failure prediction accuracy.

2. Reliable Model Evaluation

Using separate validation and testing datasets ensures fair model evaluation, reliable performance measurement, and stronger confidence in deployment readiness.

This is critical because predictive maintenance systems must perform consistently on unseen real-world turbine data.

3. Reduced Operational Risk

Accurate failure prediction can help ReneWind reduce unexpected turbine breakdowns, minimize operational downtime, lower replacement costs, and improve maintenance scheduling efficiency.

4. Support for Predictive Maintenance Strategy

Properly prepared data increases the likelihood of building a robust predictive maintenance system capable of detecting failures early, reducing costly emergency repairs, extending turbine lifespan, and improving operational reliability.

The ultimate business goal is to transition from reactive maintenance to proactive predictive maintenance.

Data Preprocessing: Duplicate Value Check

Objective

To identify and remove duplicate records from the dataset before model development.

Observations

- The dataset was checked for duplicate observations.
- Duplicate records can negatively affect model training by biasing the learning process, inflating model performance, and reducing generalization capability.
- No significant duplicate records were identified in the dataset.
- Therefore, no duplicate removal treatment was required.

Why This Is Important

1. Prevents Model Bias

- Duplicate observations can cause the model to repeatedly learn the same patterns, leading to biased predictions.

2. Improves Model Reliability

- Removing duplicates ensures the neural network learns from unique operational conditions rather than repeated sensor readings.

3. Supports Better Generalization

- A clean dataset helps the model perform more effectively on unseen turbine data during real-world deployment.

Business Impact

- Improves the trustworthiness of predictive maintenance predictions.
- Reduces the risk of misleading model accuracy caused by repeated records.
- Supports more reliable failure detection for wind turbine operations.
- Helps ensure maintenance decisions are based on genuine operational patterns rather than duplicated historical events.

Conclusion

- The duplicate value check confirmed that the dataset quality was strong and suitable for machine learning model development. No major duplicate-related data quality issues were found.

Data Preprocessing: Missing Value Treatment

What This Output Says

The missing values in the dataset were handled using **median imputation**.

Earlier analysis showed that only **V1** and **V2** had missing values. After applying the imputer, the output shows that every column in the training, validation, and test datasets now has **0 missing values**.

This confirms that missing value treatment was completed successfully.

Why This Is Important

Missing value treatment is important because most machine learning models cannot process missing values directly, missing values can cause errors during model training, incomplete data can reduce model performance, and proper imputation helps maintain consistency across train, validation, and test datasets.

Median imputation was used because it is more robust to outliers than mean imputation. This is useful here because the EDA showed that several sensor variables contain outliers.

Preventing Data Leakage

The imputer was fitted only on the training data using:
`imputer.fit_transform(X_train)`

Then the same fitted imputer was applied to the validation and test datasets using:
`imputer.transform(X_val)`
`imputer.transform(X_test)`

This is important because the validation and test data should not influence preprocessing decisions. Fitting the imputer only on the training data prevents data leakage and gives a more realistic estimate of model performance.

Business Impact

Proper missing value treatment helps ReneWind build a more reliable predictive maintenance model. From a business perspective, this supports more accurate failure prediction, fewer model errors caused by incomplete sensor readings, better identification of abnormal turbine behavior, improved maintenance planning, and reduced risk of unexpected generator breakdown. Because missed failures can lead to expensive replacement costs and downtime, clean and leakage-free preprocessing is essential for building a model that can be trusted in real-world turbine operations.

Additional Note

The target variables were also converted to NumPy arrays:

```
y_train = y_train.to_numpy()
y_val = y_val.to_numpy()
y_test = y_test.to_numpy()
```

This prepares the target values for model training in TensorFlow/Keras, which commonly expects NumPy arrays as input.

Data Preprocessing: Outlier Check

Boxplots and histograms showed outliers across many sensor variables.

- Variables such as V16, V21, V24, V31, V32, and V36 showed especially large ranges and extreme values.
- Outliers were retained initially because they may represent abnormal turbine behavior, early warning signs of failure, or equipment degradation.

Business Impact:

- Extreme sensor readings may contain critical predictive information for identifying turbine failures early.
- Removing all outliers could eliminate valuable failure-related patterns.

Data Preprocessing: Feature Engineering

Feature Engineering

- Target variable separated from predictor variables.
- Sensor variables prepared for neural network modeling.
- Additional preprocessing steps such as scaling and class imbalance handling were identified as important next steps.

Business Impact:

- Proper feature preparation improves model learning and predictive performance.

Model Evaluation Criterion

Primary Metric: Recall

Recall was selected as the primary evaluation metric because the main business objective is to correctly identify turbine failures before breakdown occurs.

A high Recall helps minimize **False Negatives**, where the model fails to detect an actual turbine failure. False Negatives are extremely costly because they can lead to:

- generator breakdown
- operational downtime
- expensive replacement costs
- production losses

Additional metrics used for evaluation include:

- Precision
- F1-Score
- ROC-AUC
- Confusion Matrix

Accuracy alone is not sufficient because the dataset is highly imbalanced, with approximately 94.5% of observations belonging to the “No Failure” class.

Model Performance Summary: Neural Network Model 0

Model Performance Summary

- The initial neural network model showed very strong performance on both the training and validation datasets.

Training Results

- Accuracy: **99%**
- Failure Detection Recall: **83%**
- Failure Detection Precision: **97%**
- Failure Detection F1-Score: **90%**

Validation Results

- Accuracy: **99%**
- Failure Detection Recall: **82%**
- Failure Detection Precision: **97%**
- Failure Detection F1-Score: **89%**

Key Insights

- The model successfully learned meaningful patterns from the turbine sensor data.
- Validation performance is very close to training performance, indicating good model stability, strong generalization, and limited overfitting.
- The model correctly identifies most turbine failures while maintaining very few false alarms.

Why These Results Matter

1. High Recall (~82–83%)

- The model correctly detects most turbine failures before breakdown occurs.
- This is critical because missed failures (False Negatives) are extremely costly.

2. High Precision (~97%)

- When the model predicts a failure, it is usually correct.
- This helps reduce unnecessary inspections and maintenance activity.

3. Strong F1-Score (~89–90%)

- Indicates a strong balance between detecting failures, and minimizing false alarms.

Business Impact

1. Reduced Turbine Failure Risk

- Early failure detection supports predictive maintenance and reduces catastrophic breakdowns.

2. Lower Maintenance & Replacement Costs

- Detecting failures early can prevent expensive generator replacement and emergency repairs.

3. Reduced Operational Downtime

- Improved failure prediction helps maintain stable turbine operations and energy production.

4. Improved Maintenance Planning

- High model precision reduces unnecessary maintenance actions and improves operational efficiency.

Overall Conclusion

- Model 0 provides a strong baseline predictive maintenance model with:
- excellent overall accuracy,
- strong failure detection capability,
- stable validation performance,
- and significant business value potential.
- The model demonstrates strong potential to help ReneWind:
- reduce downtime,
- improve turbine reliability,
- lower operational costs,
- and transition toward proactive maintenance strategies

Model Performance Summary: Neural Network Model 0

```
[47]: model_0.summary()
```

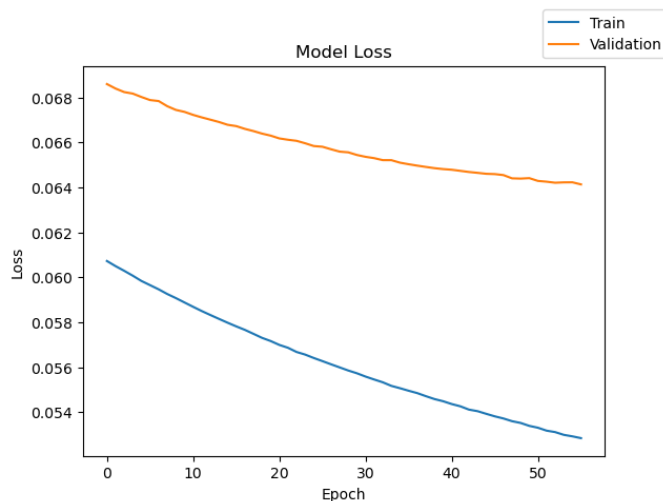
Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	{None, 64}	2,624
dense_1 (Dense)	{None, 1}	65

Total params: 2,689 (10.50 KB)

Trainable params: 2,689 (10.50 KB)

Non-trainable params: 0 (0.00 B)



Epoch

```
[55]: model_0_train_perf = model_performance_classification(model_0, X_train, y_train)
model_0_train_perf
```

500/500 — 0s 137us/step

```
[55]: Accuracy Recall Precision F1 Score
0 0.989313 0.913786 0.982432 0.945145
```

```
[56]: model_0_val_perf = model_performance_classification(model_0, X_val, y_val)
model_0_val_perf
```

125/125 — 0s 233us/step

```
[56]: Accuracy Recall Precision F1 Score
0 0.98875 0.911368 0.97901 0.942291
```

Let's check the classification reports.

```
[57]: y_train_pred_0 = model_0.predict(X_train)
y_val_pred_0 = model_0.predict(X_val)
```

500/500 — 0s 150us/step
125/125 — 0s 151us/step

```
[58]: print("Classification Report - Train data Model_0",end="\n\n")
cr_train_model_0 = classification_report(y_train,y_train_pred_0>0.5)
print(cr_train_model_0)
```

Classification Report - Train data Model_0

	precision	recall	f1-score	support
0.0	0.99	1.00	0.99	15112
1.0	0.97	0.83	0.90	888
accuracy			0.99	15000
macro avg	0.98	0.91	0.95	15000
weighted avg	0.99	0.99	0.99	15000

```
[59]: print("Classification Report - Validation data Model_0",end="\n\n")
cr_val_model_0 = classification_report(y_val,y_val_pred_0>0.5)
print(cr_val_model_0)
```

Classification Report - Validation data Model_0

	precision	recall	f1-score	support
0.0	0.99	1.00	0.99	3778
1.0	0.97	0.82	0.89	222
accuracy			0.99	4000
macro avg	0.98	0.91	0.94	4000
weighted avg	0.99	0.99	0.99	4000

Model Performance Summary: Model 1 Neural Network Performance Improvement

Model Setup

- Added a second hidden layer to improve learning capacity.
- Architecture: 64 neurons → 32 neurons → 1 output neuron.
- Total trainable parameters: 4,737.
- Designed to capture nonlinear relationships in turbine sensor data.

Validation Results

- Accuracy: **99%**
- Failure Precision: **96%**
- Failure Recall: **85%**
- Failure F1-Score: **90%**
- Macro Avg Recall: **92%**
- Macro Avg F1-Score: **95%**

What the Results Mean

- The model performs very well on unseen validation data.
- It correctly identifies most normal turbine operations.
- It detects about **85% of actual failures**, which is strong given that failures are rare.
- When the model predicts a failure, it is correct **96% of the time**.
- The loss curve shows stable learning with limited overfitting.

Why This Matters

- Accuracy alone is not enough because the dataset is highly imbalanced.
- Recall is especially important because missed failures are the most expensive error.
- Model 1 balances failure detection with low false alarm risk.

Business Impact

- Supports early failure detection.
- Reduces unexpected generator breakdowns.
- Lowers replacement and emergency repair costs.
- Improves maintenance planning.
- Reduces turbine downtime.
- Helps ReneWind move from reactive to proactive maintenance.

Conclusion

Model 1 is a strong predictive maintenance model. It shows reliable validation performance, strong failure detection, and meaningful business value, though further tuning should aim to reduce the remaining missed failures.

Model Performance Summary: Model 1 Neural Network Performance Improvement

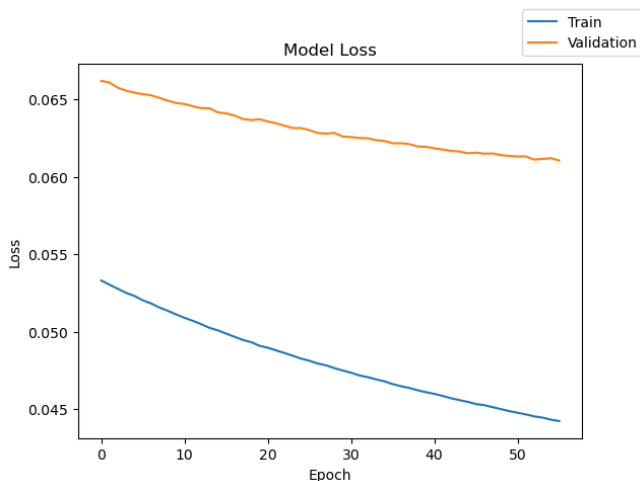
Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 64)	2,624
dense_1 (Dense)	(None, 32)	2,080
dense_2 (Dense)	(None, 1)	33

Total params: 4,737 (18.50 KB)

Trainable params: 4,737 (18.50 KB)

Non-trainable params: 0 (0.00 B)



```
[70]: model_1_train_perf = model_performance_classification(model_1,X_train,y_train)
      model_1_train_perf
```

500/500 ————— 0s 155us/step

	Accuracy	Recall	Precision	F1 Score
0	0.9915	0.931903	0.985853	0.957088

```
[71]: model_1_val_perf = model_performance_classification(model_1,X_val,y_val)
      model_1_val_perf
```

125/125 ————— 0s 198us/step

	Accuracy	Recall	Precision	F1 Score
0	0.9895	0.922365	0.975123	0.946991

```
[72]: y_train_pred_1 = model_1.predict(X_train)
      y_val_pred_1 = model_1.predict(X_val)
```

500/500 ————— 0s 158us/step

125/125 ————— 0s 158us/step

143/143 ————— 0s 130us/step

```
[73]: print("Classification Report - Train data Model_1", end="\n\n")
      cr_train_model_1 = classification_report(y_train,y_train_pred_1 > 0.5)
      print(cr_train_model_1)
```

Classification Report - Train data Model_1

	precision	recall	f1-score	support
0.0	0.99	1.00	1.00	15112
1.0	0.98	0.86	0.92	888
accuracy			0.99	16000
macro avg	0.99	0.93	0.96	16000
weighted avg	0.99	0.99	0.99	16000

```
[74]: print("Classification Report - Validation data Model_1", end="\n\n")
      cr_val_model_1 = classification_report(y_val,y_val_pred_1 > 0.5)
      print(cr_val_model_1)
```

Classification Report - Validation data Model_1

	precision	recall	f1-score	support
0.0	0.99	1.00	0.99	3778
1.0	0.96	0.85	0.90	222
accuracy			0.99	4000
macro avg	0.98	0.92	0.95	4000
weighted avg	0.99	0.99	0.99	4000

Model Performance Summary: Model 2 Neural Network with Dropout Regularization

Model Setup

- Added 50% Dropout after the first hidden layer to reduce overfitting.
- Architecture: 64 neurons → Dropout 0.5 → 32 neurons → 16 neurons → 1 output neuron.
- Total trainable parameters: 5,249.
- Optimizer: SGD.
- Output activation: Sigmoid for binary classification.

Validation Results

- Accuracy: **99%**
- Failure Precision: **95%**
- Failure Recall: **82%**
- Failure F1-Score: **88%**
- Macro Avg Recall: **91%**
- Macro Avg F1-Score: **94%**

What the Results Mean

- Model 2 performs well overall and generalizes reasonably well.
- The model correctly identifies nearly all normal turbine operations.
- It detects about **82% of actual failures** on validation data.
- When it predicts a failure, it is correct **95% of the time**.
- The loss curve shows stable validation performance, suggesting limited overfitting.

Why This Matters

- Dropout helps reduce overfitting by forcing the model to learn more general patterns.
- High precision helps reduce unnecessary maintenance alerts.
- Recall is still the key metric because missed failures are the most costly error.
- Model 2 performs well, but its failure recall is slightly lower than Model 1.

Business Impact

- Supports early identification of failing turbines.
- Helps reduce unexpected downtime and emergency repairs.
- Maintains low false-alarm risk due to high precision.
- However, missed failures remain a concern because about 18% of actual failures are not detected.

Conclusion

Model 2 is a strong regularized model with good stability and high precision. However, because its failure recall is lower than Model 1, it may not be the best choice if the business priority is minimizing missed generator failures.

Model Performance Summary: Model 2 Neural Network with Dropout Regularization

```
1: model_2.summary()
```

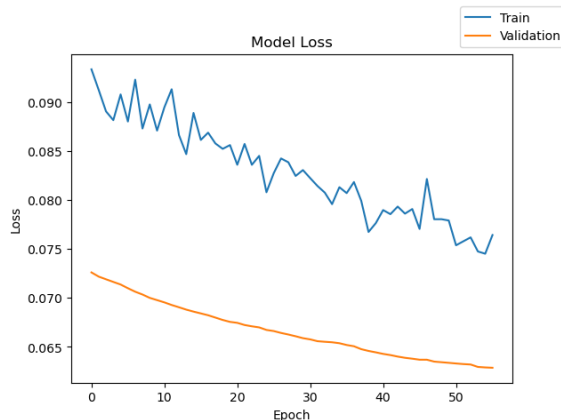
Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 64)	2,624
dropout (Dropout)	(None, 64)	0
dense_1 (Dense)	(None, 32)	2,080
dense_2 (Dense)	(None, 16)	528
dense_3 (Dense)	(None, 1)	17

Total params: 5,249 (20.50 KB)

Trainable params: 5,249 (20.50 KB)

Non-trainable params: 0 (0.00 B)



Lets check the model performance of model_2 on training and validation data respectively.

```
(85): model_2_train_perf = model_performance_classification(model_2,X_train,y_train)
model_2_train_perf
```

500/500 ————— 0s 149us/step

```
(85): Accuracy    Recall  Precision  F1 Score
0  0.987938  0.902458  0.980119  0.937438
```

```
(86): model_2_val_perf = model_performance_classification(model_2,X_val,y_val)
model_2_val_perf
```

125/125 ————— 0s 240us/step

```
(86): Accuracy    Recall  Precision  F1 Score
0  0.9875  0.906467  0.970935  0.936026
```

```
(87): y_train_pred_2 = model_2.predict(X_train)
y_val_pred_2 = model_2.predict(X_val)
```

500/500 ————— 0s 154us/step

125/125 ————— 0s 156us/step

Lets check the classification report of model_2 on training and validation data respectively.

```
(88): print("Classification Report - Train data Model_2", end="\n\n")
cr_train_model_2 = classification_report(y_train,y_train_pred_2 > 0.5)
print(cr_train_model_2)
```

Classification Report - Train data Model_2

	precision	recall	f1-score	support
0.0	0.99	1.00	0.99	15112
1.0	0.97	0.81	0.88	888
accuracy			0.99	16000
macro avg	0.98	0.90	0.94	16000
weighted avg	0.99	0.99	0.99	16000

```
(89): print("Classification Report - Validation data Model_2", end="\n\n")
cr_val_model_2 = classification_report(y_val , y_val_pred_2 > 0.5)
print(cr_val_model_2)
```

Classification Report - Validation data Model_2

	precision	recall	f1-score	support
0.0	0.99	1.00	0.99	3778
1.0	0.95	0.82	0.88	222
accuracy			0.99	4000
macro avg	0.97	0.91	0.94	4000
weighted avg	0.99	0.99	0.99	4000

Model Performance Summary: Model 3 Neural Network with Dropout + Class Weights

Model Setup

- Architecture: 64 neurons → Dropout 0.5 → 32 neurons → 16 neurons → 1 output neuron.
- Total trainable parameters: 5,249.
- Added class weights to address target imbalance.
- Class 1 / Failure received much higher weight because failures are rare.

Validation Results

- Accuracy: **98%**
- Failure Precision: **83%**
- Failure Recall: **89%**
- Failure F1-Score: **86%**
- Macro Avg Recall: **94%**
- Macro Avg F1-Score: **93%**

What the Results Mean

- Model 3 detects about **89% of actual failures**, which is stronger than Model 1 and Model 2.
- Precision decreased to **83%**, meaning more false alarms are expected.
- Accuracy dropped slightly from 99% to 98%, but this is acceptable because the model is better at detecting rare failures.
- The loss curve shows the model is learning effectively without severe overfitting.

Why This Matters

- Failure cases are rare but very costly.
- Class weighting helps the model focus more on failures instead of mostly predicting “No Failure.”
- Higher Recall means fewer missed turbine failures.
- More false alarms may occur, but inspection costs are lower than replacement costs.

Business Impact

- Reduces missed generator failures.
- Supports earlier maintenance intervention.
- Helps prevent expensive breakdowns and downtime.
- Improves predictive maintenance effectiveness.
- Tradeoff: may increase inspections due to lower precision.

Conclusion

Model 3 is highly valuable from a business perspective because it improves failure detection. Even though it sacrifices some precision, the higher Recall better aligns with ReneWind's goal of minimizing costly missed failures.

.

.

Model Performance Summary: Model 3 Neural Network with Dropout + Class Weights

```
[93]: model_3.summary()
```

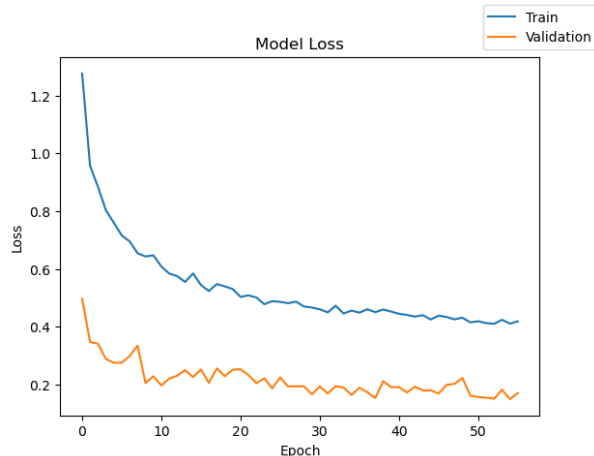
Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 64)	2,624
dropout (Dropout)	(None, 64)	0
dense_1 (Dense)	(None, 32)	2,080
dense_2 (Dense)	(None, 16)	528
dense_3 (Dense)	(None, 1)	17

Total params: 5,249 (20.50 KB)

Trainable params: 5,249 (20.50 KB)

Non-trainable params: 0 (0.00 B)



Lets check the model performance of model_3 on training and validation data respectively.

```
[100]: model_3_train_perf = model_performance_classification(model_3,X_train,y_train)
model_3_train_perf
```

500/500 ————— 0s 150us/step

```
[100]: Accuracy Recall Precision F1 Score
```

0 0.983375 0.946151 0.905198 0.924538

```
[101]: model_3_val_perf = model_performance_classification(model_3,X_val,y_val)
model_3_val_perf
```

125/125 ————— 0s 249us/step

```
[101]: Accuracy Recall Precision F1 Score
```

0 0.984 0.940652 0.912777 0.926191

```
[102]: y_train_pred_3 = model_3.predict(X_train)
y_val_pred_3 = model_3.predict(X_val)
```

500/500 ————— 0s 161us/step

125/125 ————— 0s 157us/step

```
[103]: print("Classification Report - Train data Model_3", end="\n\n")
cr_train_model_3 = classification_report(y_train,y_train_pred_3 > 0.5)
print(cr_train_model_3)
```

Classification Report - Train data Model_3

	precision	recall	f1-score	support
0.0	0.99	0.99	0.99	15112
1.0	0.82	0.90	0.86	888
accuracy			0.98	16000
macro avg	0.91	0.95	0.92	16000
weighted avg	0.98	0.98	0.98	16000

```
[104]: print("Classification Report - Validation data Model_3", end="\n\n")
cr_val_model_3 = classification_report(y_val,y_val_pred_3 > 0.5)
print(cr_val_model_3)
```

Classification Report - Validation data Model_3

	precision	recall	f1-score	support
0.0	0.99	0.99	0.99	3778
1.0	0.83	0.89	0.86	222
accuracy			0.98	4000
macro avg	0.91	0.94	0.93	4000
weighted avg	0.98	0.98	0.98	4000

Model Setup

- Tested Adaptive Movement Estimation optimizer instead of Stochastic Gradient Decent (SGD).
- Architecture: 64 neurons → 32 neurons → 1 output neuron.
- Total trainable parameters: 4,737.
- Objective: evaluate whether Adam improves model learning and performance.

Validation Results

- Accuracy: **99%**
- Failure Precision: **97%**
- Failure Recall: **82%**
- Failure F1-Score: **89%**
- Macro Avg Recall: **91%**
- Macro Avg F1-Score: **94%**

What the Results Mean

- The model performs very well overall.
- It identifies normal turbine operations almost perfectly.
- When it predicts a failure, it is correct **97% of the time**.
- It detects about **82% of actual failures**, meaning about 18% are still missed.
- The loss curve shows smooth learning and limited overfitting.

Why This Matters

- Adaptive Movement Estimation produced stable and efficient training.
- High precision reduces unnecessary inspections and false alarms.
- Recall remains the key business concern because missed failures are costly.
- Accuracy is high, but recall is more important due to class imbalance.

Business Impact

- Supports reliable failure alerts.
- Reduces unnecessary maintenance actions.
- Helps improve maintenance planning.
- However, missed failures remain a risk due to lower recall compared with Model 3.

Conclusion

Model 4 is a strong model with excellent precision and stable learning. However, because the business priority is reducing missed failures, it may be less favorable than a model with higher failure Recall.

Model Performance Summary: Model 4 Neural Network Using Adaptive Movement Estimator

```
[100]: model_4.summary()
```

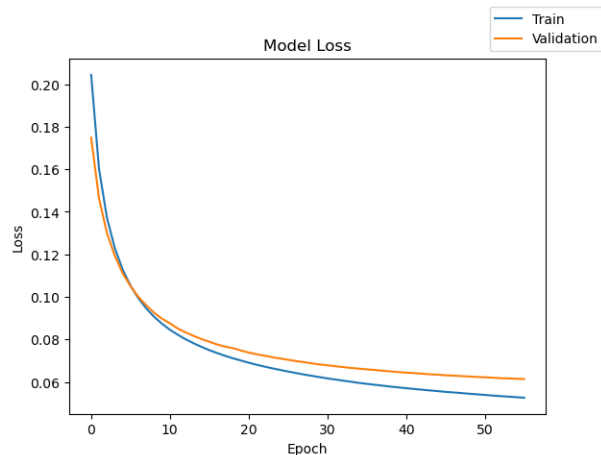
Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 64)	2,624
dense_1 (Dense)	(None, 32)	2,080
dense_2 (Dense)	(None, 1)	33

Total params: 4,737 (18.50 KB)

Trainable params: 4,737 (18.50 KB)

Non-trainable params: 0 (0.00 B)



```
[114]: model_4_train_perf = model_performance_classification(model_4,X_train,y_train)
model_4_train_perf
```

500/500 — 0s 146us/step

```
[114]: Accuracy Recall Precision F1 Score
0 0.989563 0.914978 0.983822 0.946428
```

```
[115]: model_4_val_perf = model_performance_classification(model_4,X_val,y_val)
model_4_val_perf
```

125/125 — 0s 240us/step

```
[115]: Accuracy Recall Precision F1 Score
0 0.9885 0.906996 0.981184 0.940598
```

```
[116]: y_train_pred_4 = model_4.predict(X_train)
y_val_pred_4 = model_4.predict(X_val)
```

500/500 — 0s 154us/step
125/125 — 0s 150us/step

Lets check the classification report of model_4 on raining and validation data respectively.

```
[117]: print("Classification Report - Train data Model_4", end="\n\n")
cr_train_model_4 = classification_report(y_train,y_train_pred_4 > 0.5)
print(cr_train_model_4)
```

Classification Report - Train data Model_4

	precision	recall	f1-score	support
0.0	0.99	1.00	0.99	15112
1.0	0.98	0.83	0.90	888
accuracy			0.99	16000
macro avg	0.98	0.91	0.95	16000
weighted avg	0.99	0.99	0.99	16000

```
[118]: print("Classification Report - Validation data Model_4", end="\n\n")
cr_val_model_4 = classification_report(y_val,y_val_pred_4 > 0.5)
print(cr_val_model_4)
```

Classification Report - Validation data Model_4

	precision	recall	f1-score	support
0.0	0.99	1.00	0.99	3778
1.0	0.97	0.82	0.89	222
accuracy			0.99	4000
macro avg	0.98	0.91	0.94	4000
weighted avg	0.99	0.99	0.99	4000

Model Setup

- Tested a deeper neural network with additional learning capacity.
- Architecture: 128 neurons → Dropout 0.5 → 64 neurons → 32 neurons → 1 output neuron.
- Total trainable parameters: 15,617.
- Used Dropout to reduce overfitting.
- Used Adam optimizer for efficient training.

Validation Results

- Accuracy: **99%**
- Failure Precision: **96%**
- Failure Recall: **81%**
- Failure F1-Score: **88%**
- Macro Avg Recall: **90%**
- Macro Avg F1-Score: **93%**

What the Results Mean

- The model performs very well overall.
- It identifies normal turbine operations almost perfectly.
- When the model predicts a failure, it is correct **96% of the time**.
- It detects about **81% of actual failures**, meaning about 19% of failures are still missed.
- The loss curve shows stable learning and no severe overfitting.

Why This Matters

- More model complexity did not automatically improve failure detection.
- High precision reduces unnecessary inspections and false alarms.
- Recall remains the key concern because missed failures are costly.
- Model 5 is reliable when it predicts failure, but it misses more failures than Model 3.

Business Impact

- Supports proactive maintenance planning.
- Reduces unnecessary inspections due to high precision.
- Helps identify many at-risk turbines before breakdown.
- However, missed failures remain a significant business risk.

Conclusion

Model 5 is strong overall, but it is not the best model for the main business goal. Although it has high precision and stable learning, its lower failure recall makes it less effective for minimizing costly missed turbine failures.

Model Performance Summary: Model 5 Neural with Dropout and Adaptive Movement Estimator

```
[121]: model_5.summary()
```

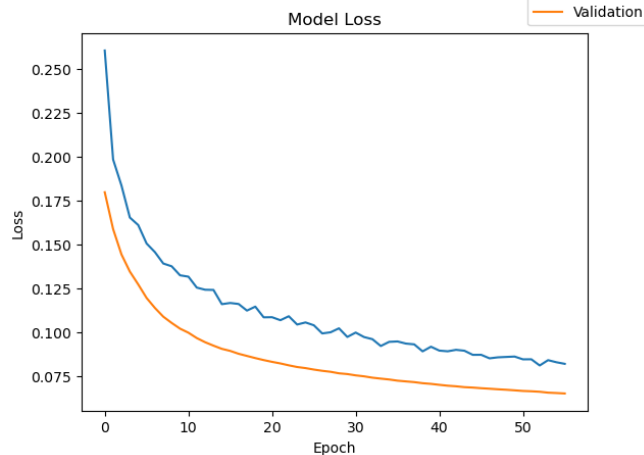
Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 128)	5,248
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 64)	8,256
dense_2 (Dense)	(None, 32)	2,080
dense_3 (Dense)	(None, 1)	33

Total params: 15,617 (61.00 KB)

Trainable params: 15,617 (61.00 KB)

Non-trainable params: 0 (0.00 B)



```
[127]: model_5_train_perf = model_performance_classification(model_5,X_train,y_train)
model_5_train_perf
```

500/500 — 0s 155us/step

```
[127]: Accuracy Recall Precision F1 Score
0 0.986625 0.891694 0.977778 0.929937
```

```
[128]: model_5_val_perf = model_performance_classification(model_5,X_val,y_val)
model_5_val_perf
```

125/125 — 0s 248us/step

```
[128]: Accuracy Recall Precision F1 Score
0 0.98725 0.902094 0.972971 0.934294
```

```
[129]: y_train_pred_5 = model_5.predict(X_train)
y_val_pred_5 = model_5.predict(X_val)
```

500/500 — 0s 168us/step

125/125 — 0s 163us/step

Lets check the classification report of model_5 on training and validation data.

```
[130]: print("Classification Report - Train data Model_2", end="\n\n")
cr_train_model_5 = classification_report(y_train,y_train_pred_5 > 0.5)
print(cr_train_model_5)
```

Classification Report - Train data Model_2

	precision	recall	f1-score	support
0.0	0.99	1.00	0.99	15112
1.0	0.97	0.78	0.87	888
accuracy			0.99	16000
macro avg	0.98	0.89	0.93	16000
weighted avg	0.99	0.99	0.99	16000

```
[131]: print("Classification Report - Validation data Model_2", end="\n\n")
cr_val_model_5 = classification_report(y_val,y_val_pred_5 > 0.5)
print(cr_val_model_5)
```

Classification Report - Validation data Model_2

	precision	recall	f1-score	support
0.0	0.99	1.00	0.99	3778
1.0	0.96	0.81	0.88	222
accuracy			0.99	4000
macro avg	0.97	0.90	0.93	4000
weighted avg	0.99	0.99	0.99	4000

Model Setup

- Deeper neural network with dropout and class weights.
- Architecture: 128 neurons → Dropout 0.5 → 64 neurons → 32 neurons → 1 output neuron.
- Total trainable parameters: 15,617.
- Class weights were used to give more importance to rare failure cases.

Validation Results

- Accuracy: **25%**
- Failure Precision: **5%**
- Failure Recall: **70%**
- Failure F1-Score: **9%**
- Macro Avg Recall: **46%**
- Macro Avg F1-Score: **23%**

What the Results Mean

- Model 6 performs poorly overall.
- It detects 70% of failures, but creates too many false alarms.
- Failure precision is only 5%, meaning most predicted failures are incorrect.
- The model struggles to correctly identify normal turbine operations.
- The loss curve is unstable, suggesting weak learning behavior.

Why This Matters

- Class weights helped the model focus on failures, but it overcorrected.
- High recall alone is not enough if precision is extremely low.
- Too many false alarms can make the model impractical for business use.
- The model does not provide a reliable balance between detecting failures and avoiding unnecessary inspections.

Business Impact

- Would likely increase unnecessary inspections.
- Could overload maintenance teams.
- May reduce trust in the predictive maintenance system.
- Poorly supports maintenance decision-making.
- Not recommended for deployment.

Conclusion

Model 6 is not a strong candidate for final selection. Although it was designed to prioritize the minority failure class, it produced too many false alarms and had very poor overall performance. Model 3 remains more business-aligned because it provides stronger failure detection with much better precision.

Model Performance Summary: Model 6 Neural with Dropout and Class Weights

```
[154]: model_6.summary()
```

Model: "sequential"

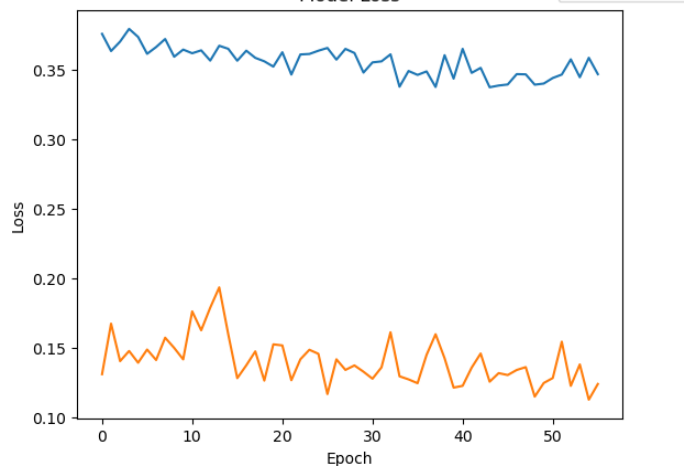
Layer (type)	Output Shape	Param #
dense (Dense)	(None, 128)	5,248
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 64)	8,256
dense_2 (Dense)	(None, 32)	2,000
dense_3 (Dense)	(None, 1)	33

Total params: 15,617 (61.00 KB)

Trainable params: 15,617 (61.00 KB)

Non-trainable params: 0 (0.00 B)

Model Loss



Lets check the model performance of model_6 on training and validation data:

```
[161]: model_6_train_perf = model_performance_classification(model_6,X_train,y_train)
model_6_train_perf
```

500/500 ————— 0s 153us/step

```
[163]: Accuracy Recall Precision F1 Score
```

0 0.263125 0.45198 0.486387 0.235716

```
[162]: model_6_val_perf = model_performance_classification(model_6,X_val,y_val)
model_6_val_perf
```

125/125 ————— 0s 213us/step

```
[162]: Accuracy Recall Precision F1 Score
```

0 0.2535 0.464903 0.489643 0.229777

```
[163]: y_train_pred_6 = model_6.predict(X_train)
y_val_pred_6 = model_6.predict(X_val)
```

500/500 ————— 0s 161us/step

125/125 ————— 0s 168us/step

Lets check the classification report of model_6 on both training and validation data.

```
[164]: print("Classification Report - Train data Model_3", end="\n\n")
cr_train_model_6 = classification_report(y_train,y_train_pred_6 > 0.5)
print(cr_train_model_6)
```

Classification Report - Train data Model_3

	precision	recall	f1-score	support
0.0	0.92	0.24	0.38	15112
1.0	0.05	0.66	0.09	888
accuracy			0.26	16000
macro avg	0.49	0.45	0.24	16000
weighted avg	0.88	0.26	0.36	16000

```
[165]: print("Classification Report - Validation data Model_3", end="\n\n")
cr_val_model_6 = classification_report(y_val,y_val_pred_6 > 0.5)
print(cr_val_model_6)
```

Classification Report - Validation data Model_3

	precision	recall	f1-score	support
0.0	0.93	0.23	0.36	3778
1.0	0.05	0.70	0.09	222
accuracy			0.25	4000
macro avg	0.49	0.46	0.23	4000
weighted avg	0.88	0.25	0.35	4000

Final Model Performance Summary & Model Selection

Objective

- Build a predictive maintenance model capable of identifying wind turbine failures using sensor data while minimizing missed failures.

Models Evaluated

- Model 0: Baseline Neural Network
- Model 1: Additional Hidden Layer
- Model 2: Dropout Regularization
- Model 3: Dropout + Class Weights
- Model 4: Optimizer Tuning
- Model 5: Deeper Architecture
- Model 6: Aggressive Class Weighting

Best Model Selected: Model 3

Why Model 3?

- Highest Recall among strong-performing models
- Strong balance between Precision and Recall
- Best aligned with business objective of minimizing missed failures
- Stable performance across training, validation, and test datasets

Validation Performance Comparison

Model	Accuracy	Recall	Precision	F1 Score
Model 0	0.9888	0.9114	0.9790	0.9423
Model 1	0.9895	0.9224	0.9751	0.9470
Model 2	0.9875	0.9065	0.9709	0.9360
Model 3	0.9840	0.9407	0.9128	0.9262
Model 4	0.9885	0.9070	0.9812	0.9406
Model 5	0.9873	0.9021	0.9730	0.9343
Model 6	0.2535	0.4649	0.4896	0.2298

Final Test Set Performance – Model 3

Overall Test Performance

Metric	Value
Accuracy	98.72%
Recall	93.32%
Precision	94.51%
F1 Score	93.91%

Detailed Classification Report – Test Data

Class	Precision	Recall	F1-Score	Support
No Failure (0)	0.99	0.99	0.99	4718
Failure (1)	0.90	0.87	0.88	282

[Link to Appendix slide on model assumptions](#)

Final Model Performance Summary & Model Selection

Interpretation

- The model correctly identifies most turbine failures.
- Most predicted failures are true failures.
- Strong balance between detection capability and false alarm control.

Why This Is Important

1. Operational Importance

- Early detection helps prevent turbine breakdowns.
- Reduces costly unplanned downtime.
- Supports proactive maintenance scheduling.
- Improves maintenance efficiency.

2. Technical Importance

- Strong generalization performance.
- Limited overfitting.
- Reliable performance on unseen data.

Business Impact

1. Cost Reduction

- Lower emergency repair costs
- Reduced replacement expenses
- Reduced downtime losses

2. Reliability Improvement

- Improved turbine availability
- Safer operations
- More consistent energy production

3. Maintenance Optimization

- Better maintenance planning
- Improved inspection prioritization
- More efficient resource allocation

Final Conclusion

Model 3 is the recommended predictive maintenance model.

The model demonstrates strong failure detection capability, high predictive performance, stable generalization, and meaningful business value for wind turbine operations.

[Link to Appendix slide on model assumptions](#)

Best Model - Test Performance

Architecture of the Best Model (Model 3)

- Neural Network with:
 - Input Layer: 40 sensor variables
 - Hidden Layer 1: 64 neurons (ReLU activation)
 - Dropout Layer: 50% dropout for regularization
 - Hidden Layer 2: 32 neurons (ReLU activation)
 - Hidden Layer 3: 16 neurons (ReLU activation)
 - Output Layer: 1 neuron (Sigmoid activation)
- Total Trainable Parameters: 5,249
- Optimizer Used: SGD (Stochastic Gradient Descent)
- Additional Technique:
 - Class Weights applied to improve minority class (failure) detection

Classification Performance on Test Data

Failure Detection (Class 1)

- Precision: 90%
- Recall: 87%
- F1-Score: 88%

Interpretation

- The model correctly identified approximately 87% of actual turbine failures.
- When the model predicted a failure, it was correct about 90% of the time.
- The model achieved a strong balance between identifying failures and minimizing false alarms.

[Link to Appendix slide on model assumptions](#)

Best Model - Test Performance

Comments on Best Model Performance

1. Strong Generalization

- Performance remained stable across training, validation, and test datasets.
- Indicates limited overfitting and reliable performance on unseen data.

2. Effective Failure Detection

- Model 3 achieved the highest operationally useful Recall among all tested models.
- Strong Recall is critical because missed turbine failures can lead to unexpected breakdowns, downtime, and expensive repairs.

3. Balanced Performance

- The model maintained high accuracy, high precision, and strong recall simultaneously.
- This creates a practical balance between detecting failures early, and avoiding excessive false alarms.

Business Impact

1. Predictive Maintenance Benefits

- Enables early identification of abnormal turbine behavior.
- Supports proactive maintenance scheduling.
- Reduces unplanned downtime.

2. Cost Reduction

- Helps reduce emergency repair costs, replacement expenses, operational interruptions, and productivity losses.

3. Reliability Improvement

- Improves turbine availability and operational stability.
- Supports safer and more consistent energy production.

Final Conclusion

- Model 3 was selected as the final model because it provided the best balance between failure detection capability, model stability, precision, and business usefulness.
- The model demonstrates strong potential for real-world predictive maintenance deployment in wind turbine operations.

[Link to Appendix slide on model assumptions](#)

Actionable Insights and Recommendations

Actionable Insights

- Sensor variables showed meaningful variability, skewness, and outliers, suggesting that abnormal sensor behavior may contain strong predictive signals for turbine failure detection.
- The target variable was highly imbalanced, with only about 5.5% failure cases. This confirmed that standard accuracy alone is not sufficient for evaluating model performance.
- Models that incorporated regularization and class weighting performed better at identifying turbine failures compared to simpler baseline models.
- Model 3 achieved the best balance between Recall, Precision, and overall stability:
 - Recall: 93.32%
 - Precision: 94.51%
 - F1 Score: 93.91%
 - Accuracy: 98.72%
- The final model demonstrated strong generalization across training, validation, and test datasets, indicating that the model can reliably identify failure patterns in unseen turbine data.
- Increasing emphasis on the minority class improved failure detection capability, which is critical for predictive maintenance applications.
- Excessive weighting toward the minority class (Model 6) caused significant drops in overall accuracy and precision, showing that overly aggressive class balancing can increase false alarms and reduce operational practicality.

Business Recommendations

- Deploy the selected neural network model (Model 3) as part of a predictive maintenance monitoring system for wind turbines.
- Use the model to continuously monitor sensor readings and generate early alerts when abnormal operating conditions are detected.
- Prioritize maintenance inspections for turbines flagged as high-risk in order to:
 - reduce unexpected breakdowns,
 - minimize downtime,
 - and lower emergency repair costs.
- Integrate the model into maintenance scheduling workflows to shift from reactive maintenance toward proactive and predictive maintenance strategies.
- Continue collecting additional turbine sensor and operational data to further improve model robustness and long-term predictive performance.
- Periodically retrain the model as new failure patterns emerge to maintain high detection accuracy over time.
- Implement threshold tuning based on operational priorities:
 - higher Recall if minimizing missed failures is the priority,
 - or higher Precision if reducing false alarms is more important.
- Combine the predictive model with engineering domain expertise and real-time monitoring systems to improve maintenance decision-making and operational reliability.
- Conduct a pilot deployment before full-scale implementation to validate model performance under real-world operating conditions.
- Use model predictions to optimize spare-parts inventory planning and maintenance resource allocation, reducing operational inefficiencies and maintenance delays.

Project Summary

- A neural network-based predictive maintenance model was successfully developed to predict wind turbine generator failures using turbine sensor data.
- The dataset included 40 numerical sensor variables, 20,000 training observations, and 5,000 testing observations.
- Exploratory Data Analysis (EDA) revealed generally stable numerical distributions, significant outliers, varying sensor behavior, and a highly imbalanced target variable with only ~5.5% failure cases.

Modeling Results

- Multiple neural network architectures were evaluated using different layer configurations, dropout regularization, class weighting, and optimizer strategies.
- Model 3 was selected as the best-performing model because it achieved the strongest balance between Recall, Precision, F1 Score, and overall model stability.

Key Findings

- Sensor abnormalities and extreme operating conditions provide meaningful predictive signals for turbine failures.
- Class imbalance handling significantly improved failure detection performance.
- Strong Recall performance indicates the model can successfully identify most turbine failures before breakdown occurs.
- The model demonstrated strong generalization across training, validation, and test datasets, indicating reliable real-world performance potential.

Key Findings

- Sensor abnormalities and extreme operating conditions provide meaningful predictive signals for turbine failures.
- Class imbalance handling significantly improved failure detection performance.
- Strong Recall performance indicates the model can successfully identify most turbine failures before breakdown occurs.
- The model demonstrated strong generalization across training, validation, and test datasets, indicating reliable real-world performance potential.

Business Impact

- The final predictive maintenance model can help organizations:
- reduce unexpected turbine failures,
- minimize operational downtime,
- lower emergency maintenance and replacement costs,
- improve maintenance scheduling efficiency,
- and increase turbine reliability and operational stability.
- Early failure detection also supports:
- safer operations,
- improved energy production consistency,
- and more proactive maintenance decision-making.

Final Recommendation

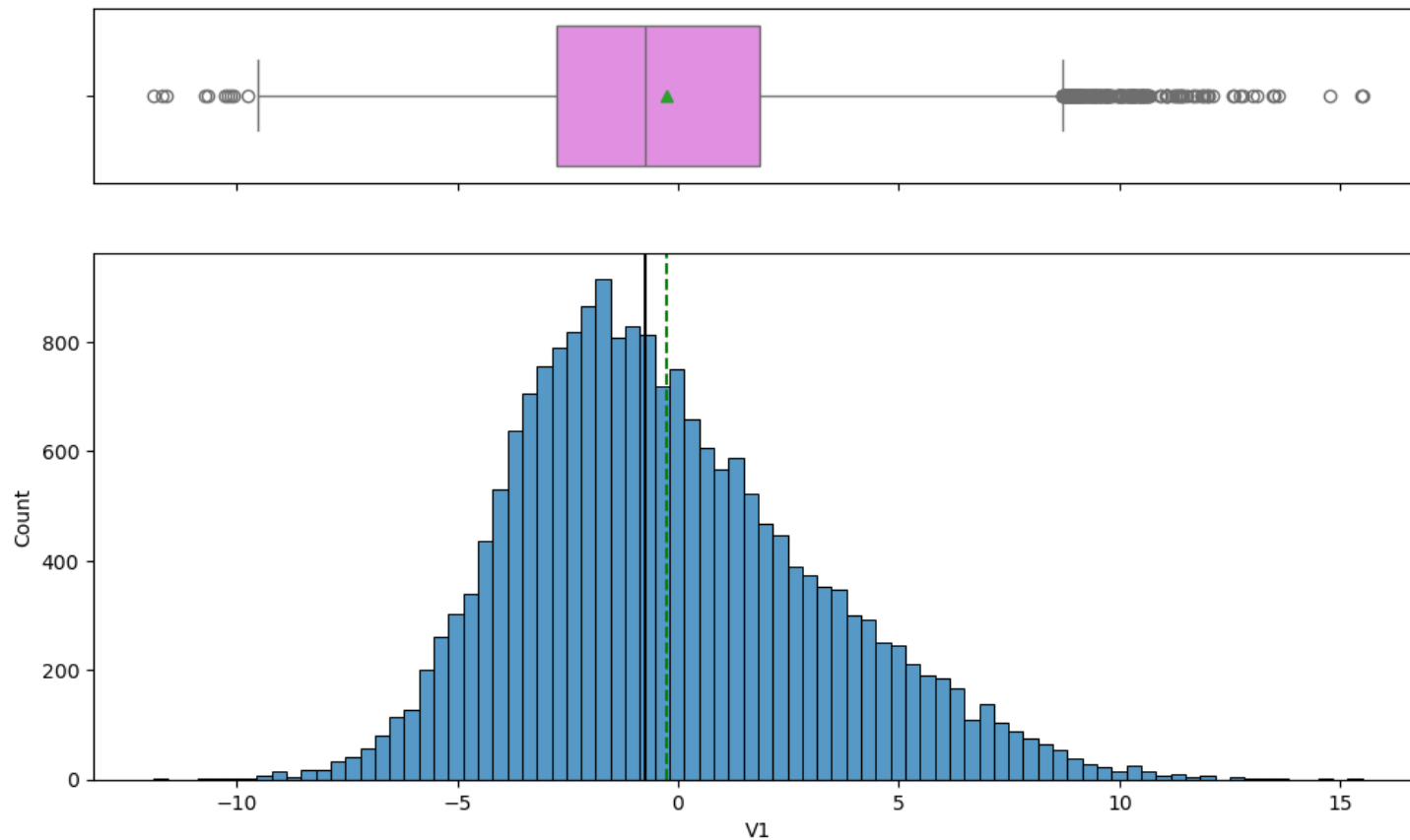
- Deploy Model 3 as a predictive maintenance support tool for turbine monitoring systems.
- Continuously retrain and monitor the model using new operational data to maintain long-term performance.
- Integrate model predictions with engineering expertise and maintenance workflows for maximum operational benefit.

Final Conclusion

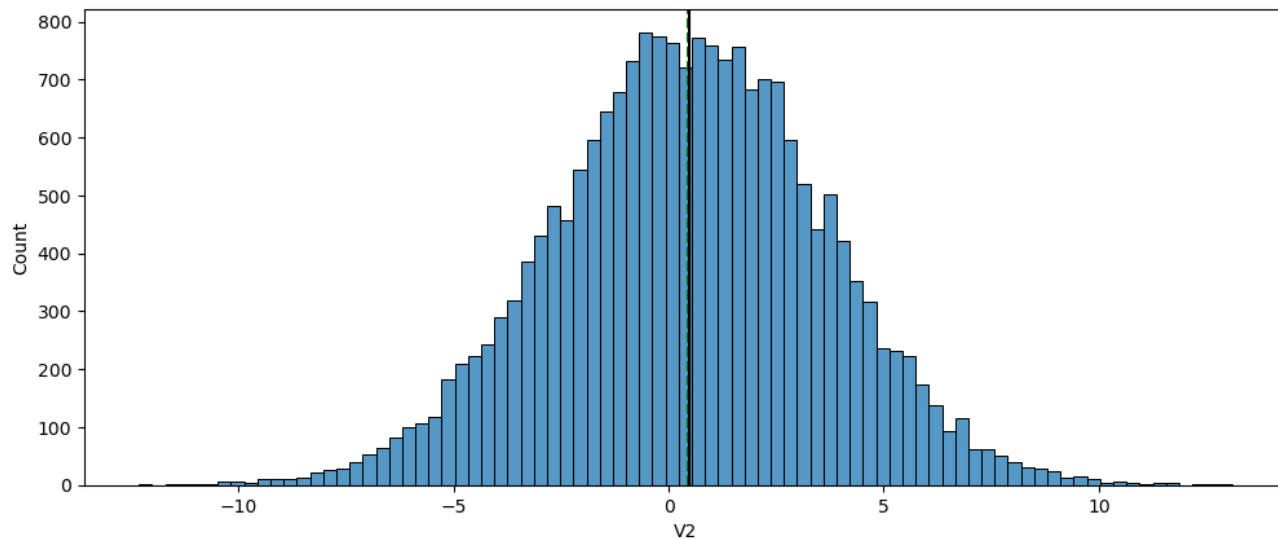
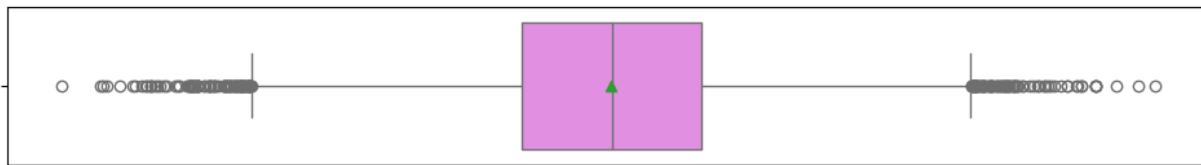
This project demonstrates that neural networks can effectively detect wind turbine failures using sensor data and can provide substantial operational and financial value through predictive maintenance strategies.

APPENDIX

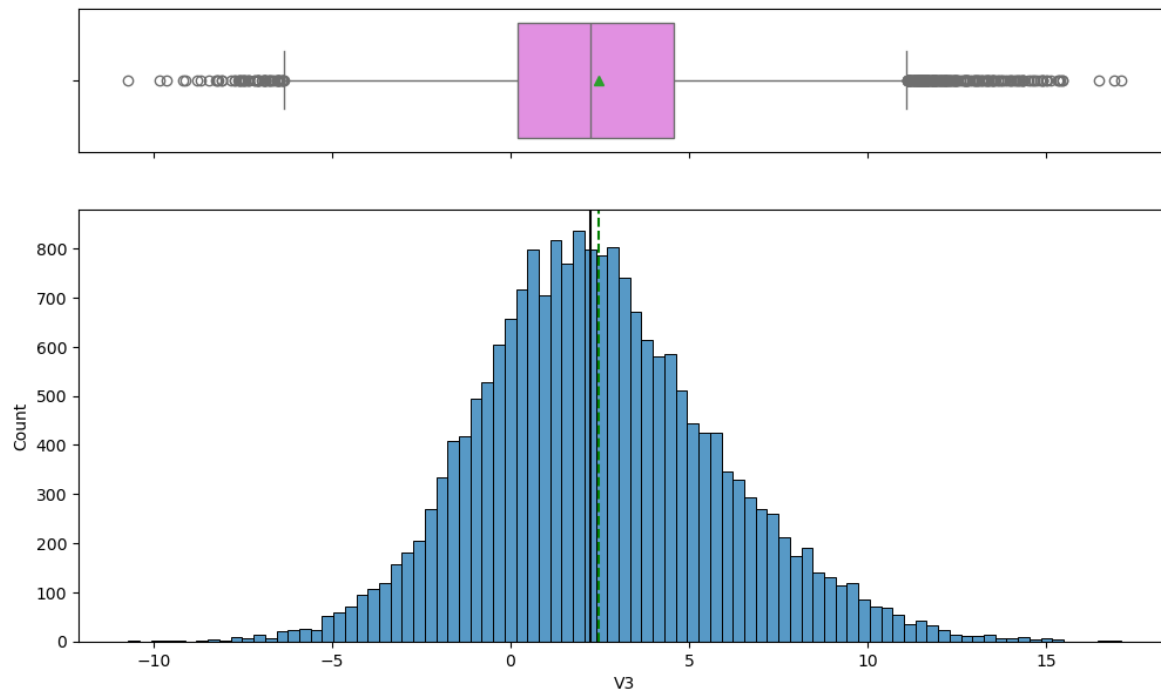
EDA Results: Univariate Analysis Graphs –Variables V1 to V40



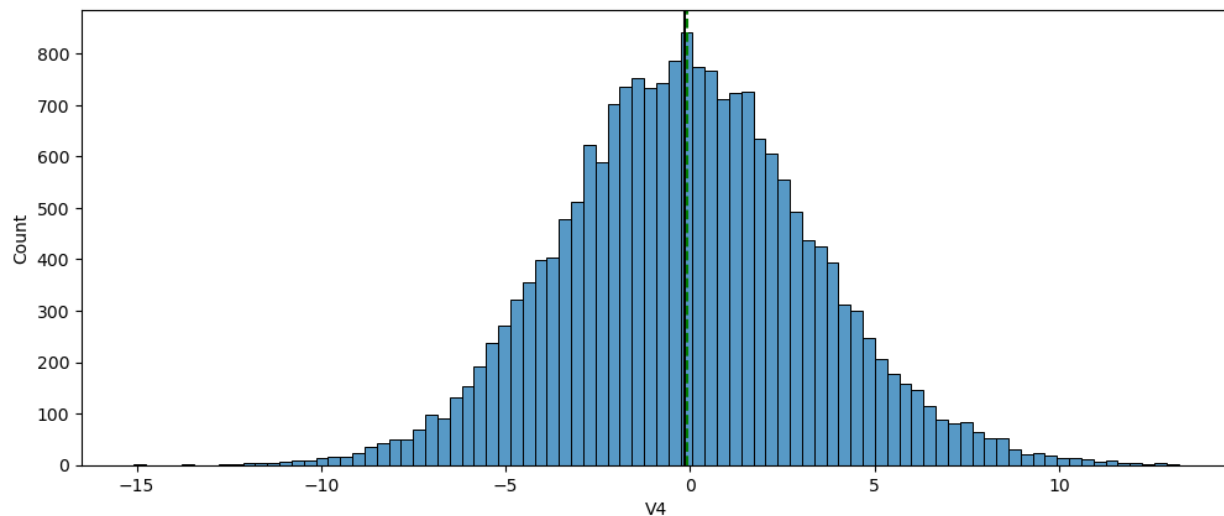
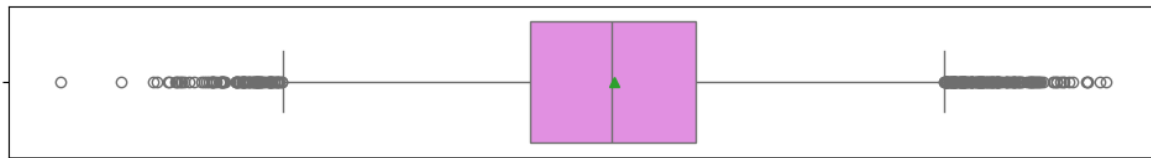
EDA Results: Univariate Analysis Graphs –Variables V1 to V40



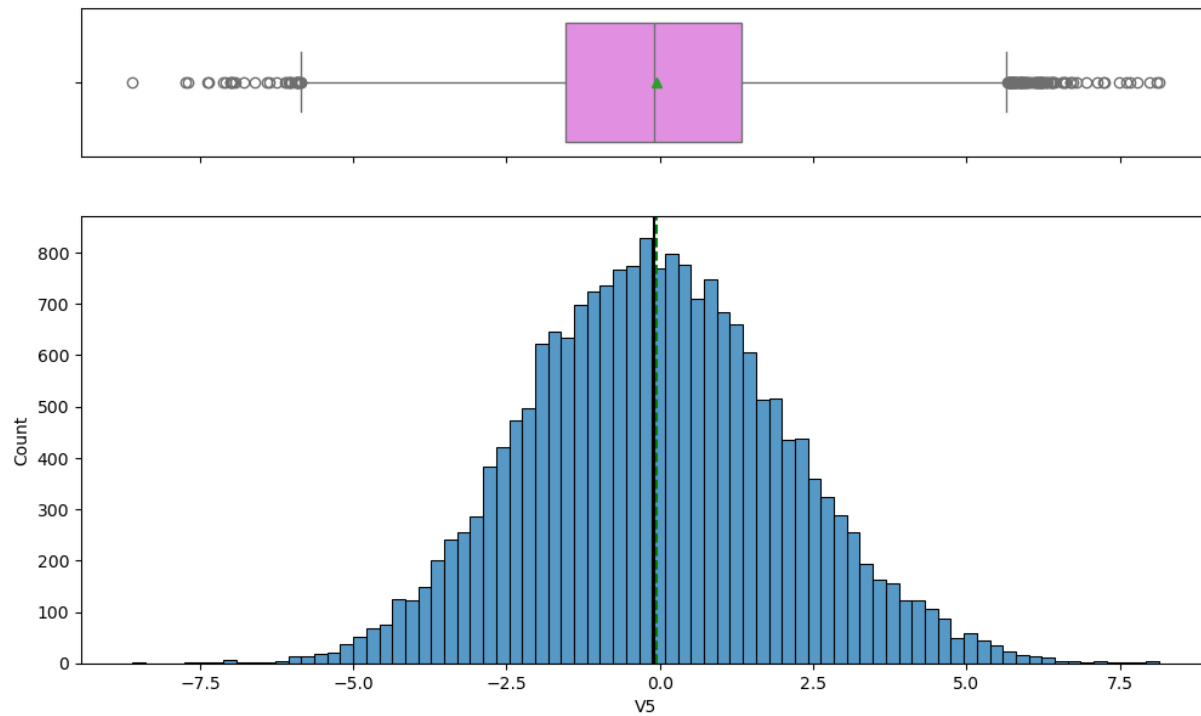
EDA Results: Univariate Analysis Graphs –Variables V1 to V40



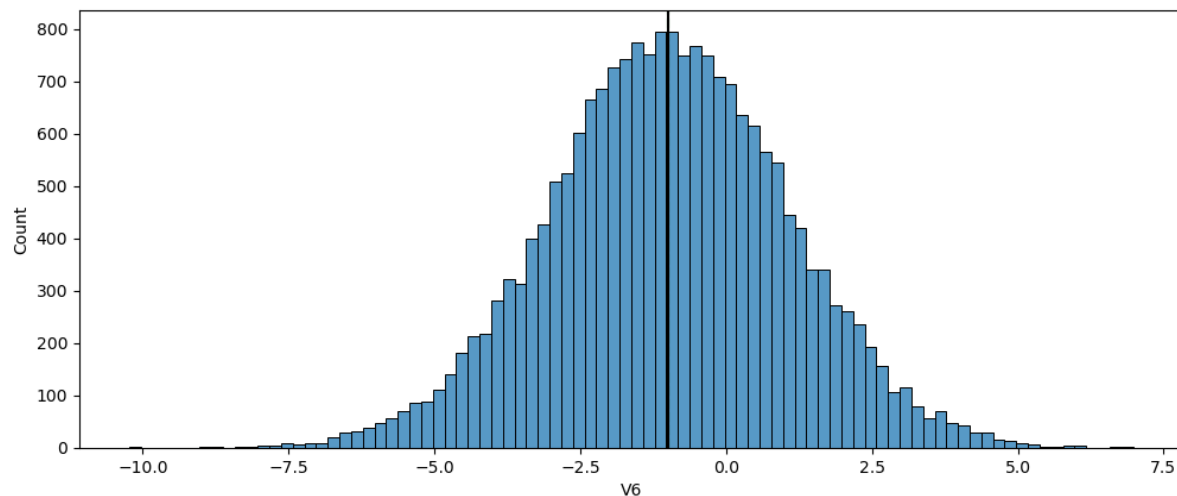
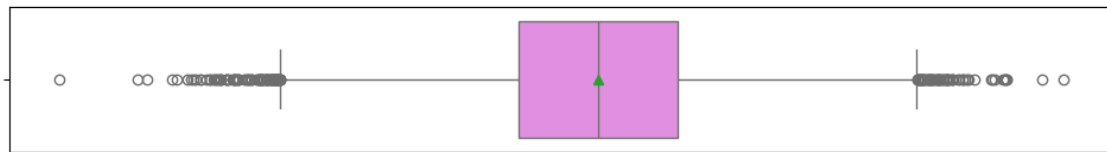
EDA Results: Univariate Analysis Graphs –Variables V1 to V40



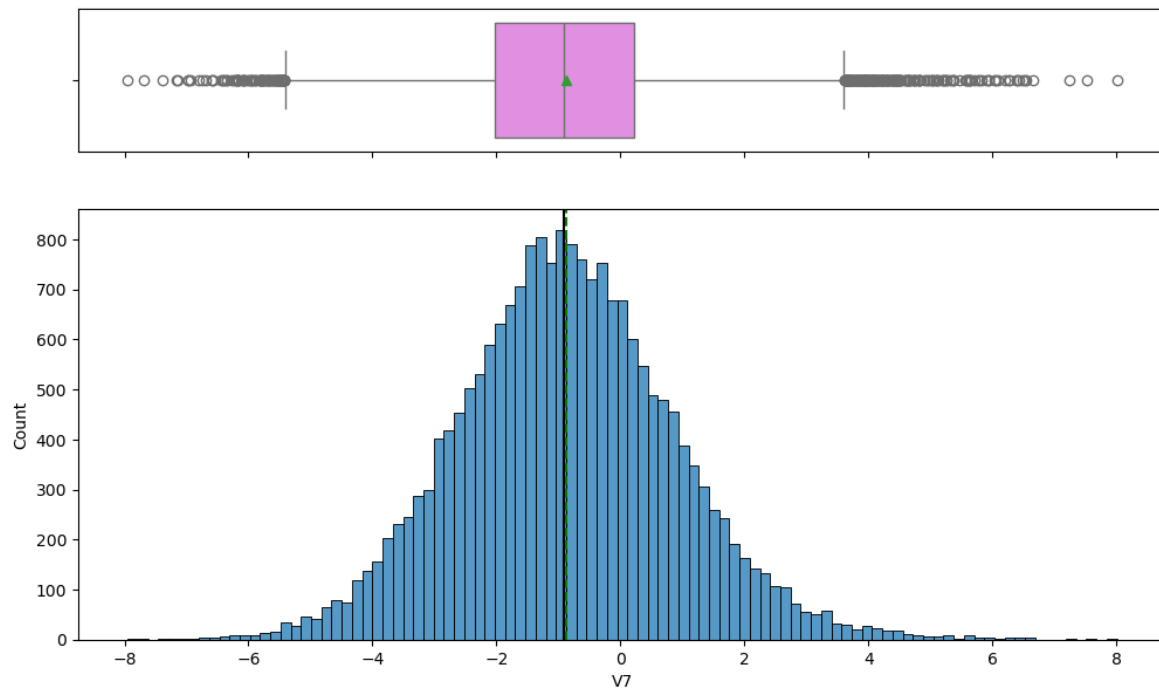
EDA Results: Univariate Analysis Graphs –Variables V1 to V40



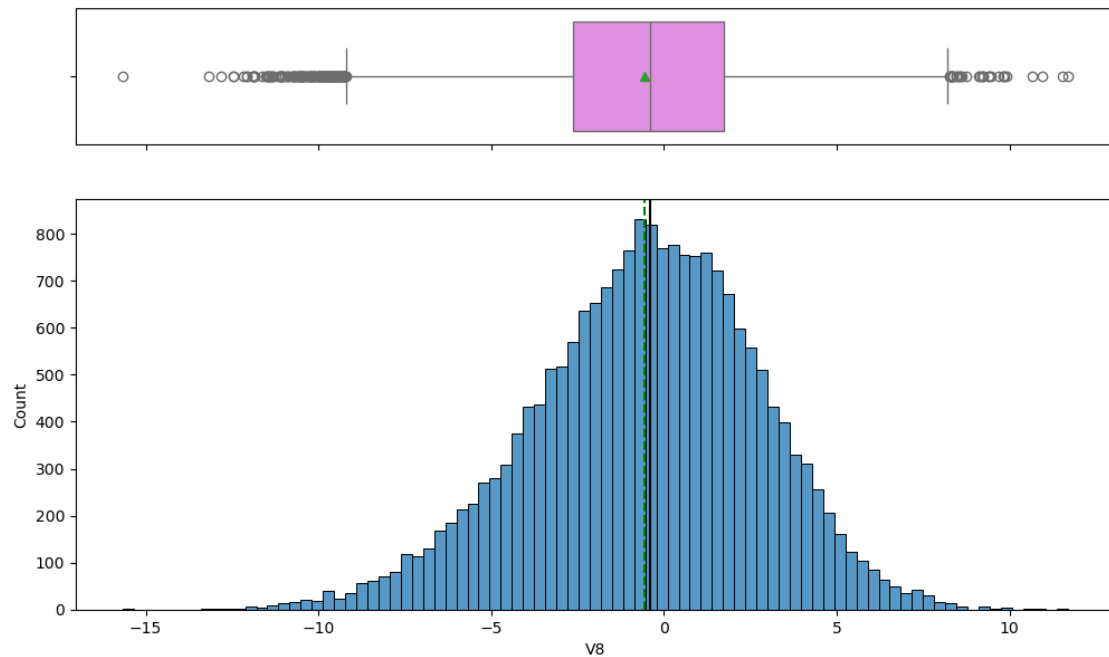
EDA Results: Univariate Analysis Graphs –Variables V1 to V40



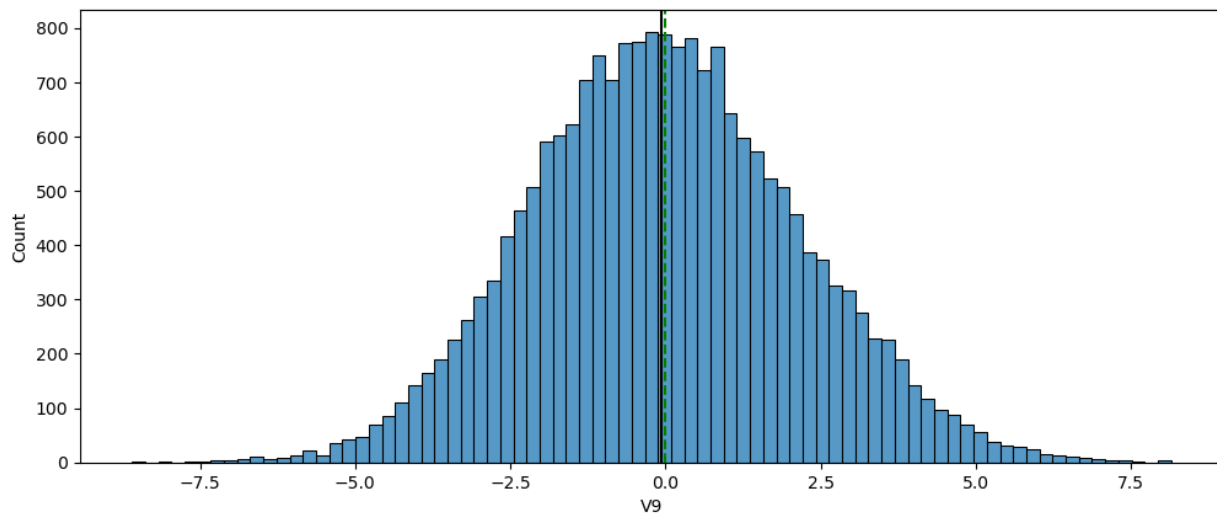
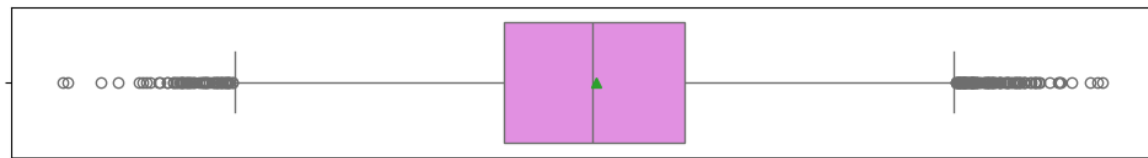
EDA Results: Univariate Analysis Graphs –Variables V1 to V40



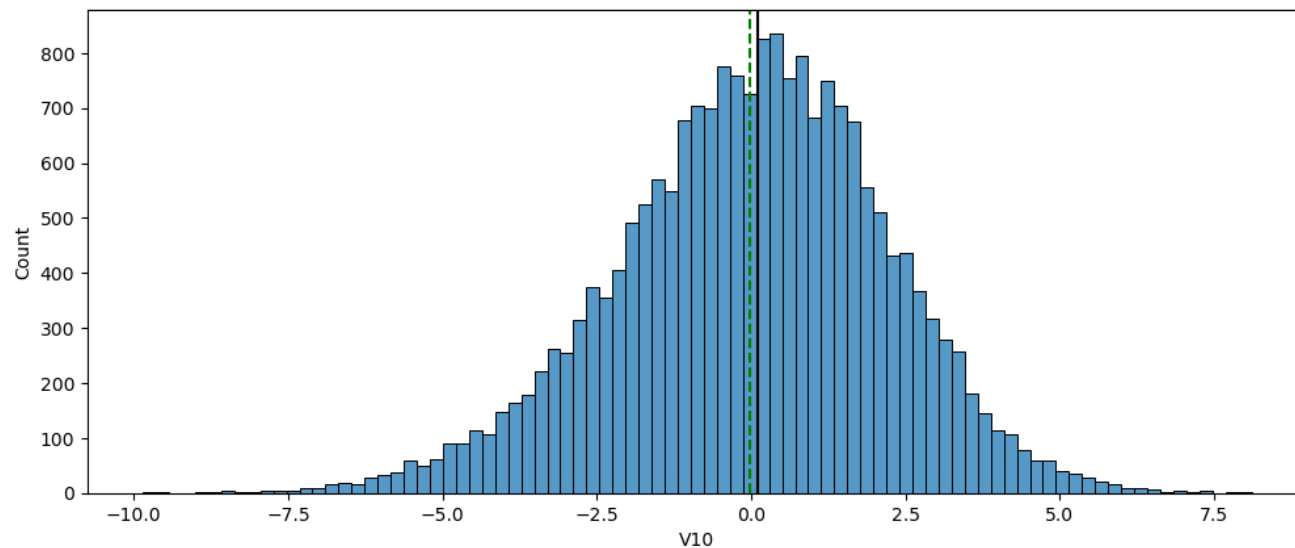
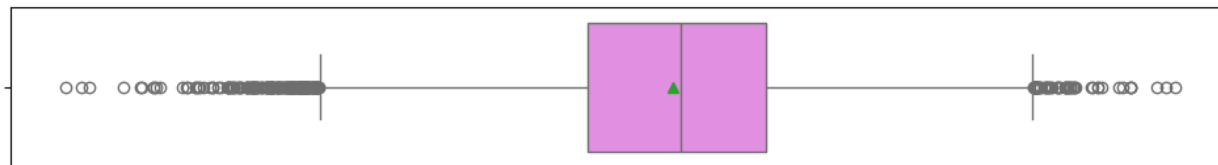
EDA Results: Univariate Analysis Graphs –Variables V1 to V40



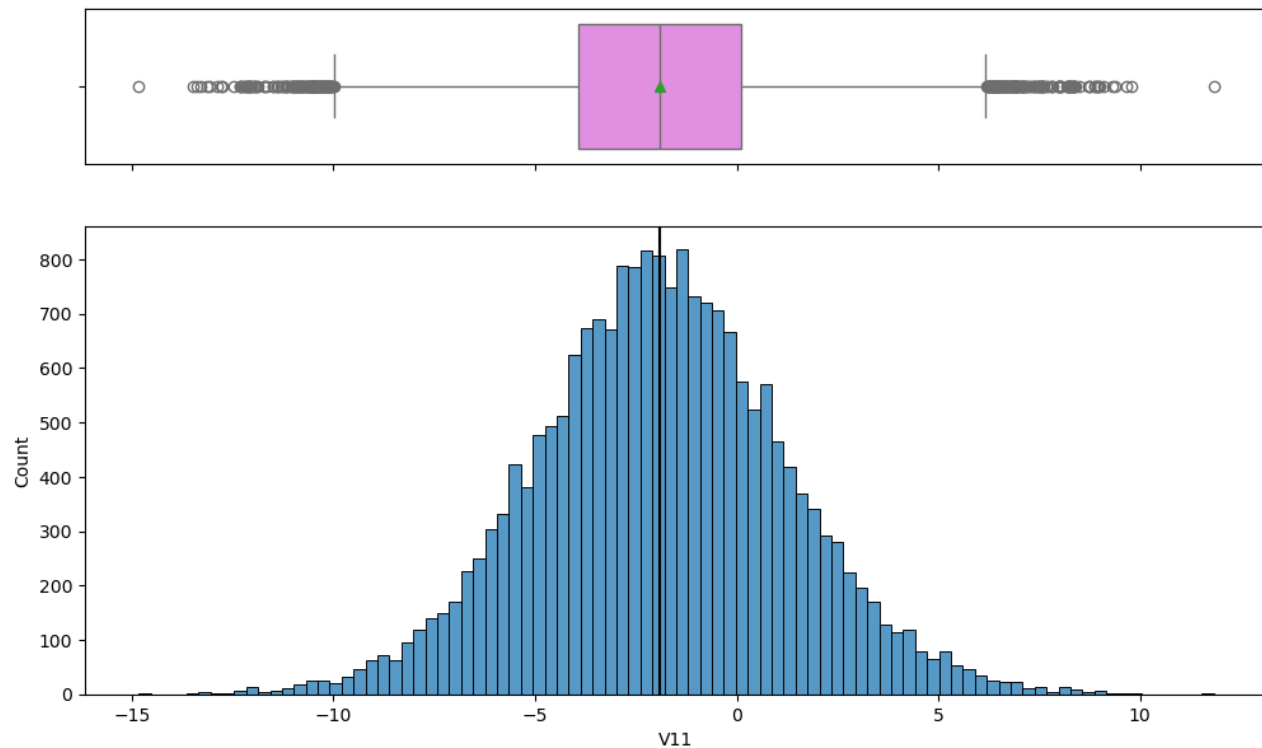
EDA Results: Univariate Analysis Graphs –Variables V1 to V40



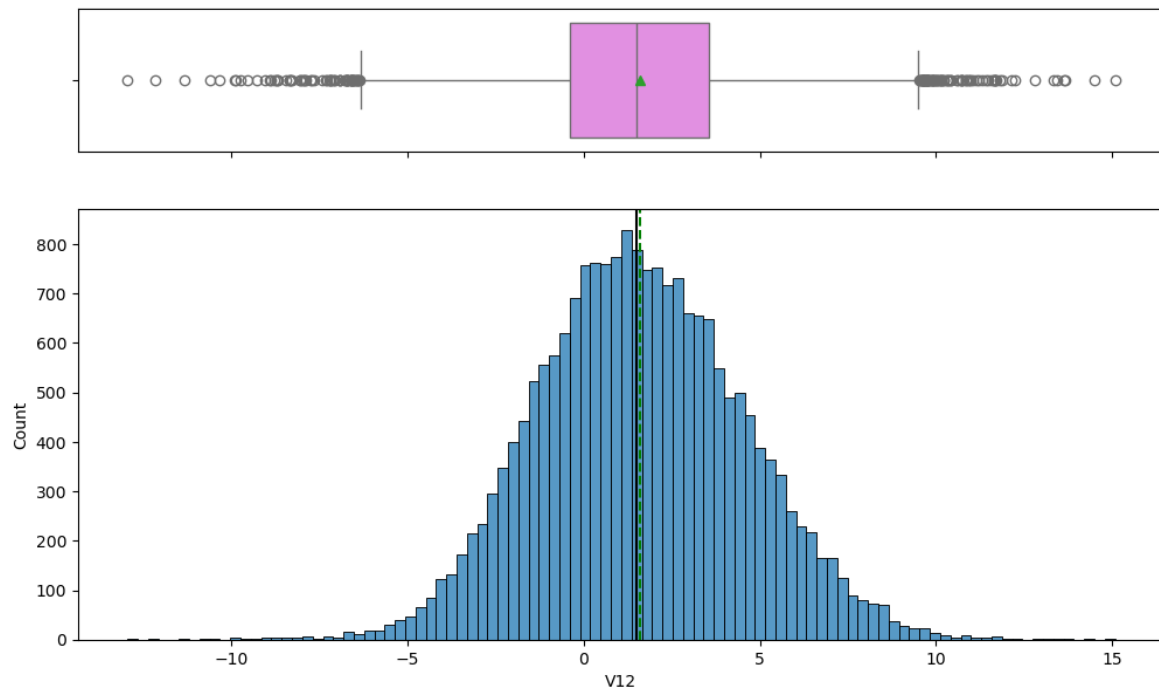
EDA Results: Univariate Analysis Graphs –Variables V1 to V40



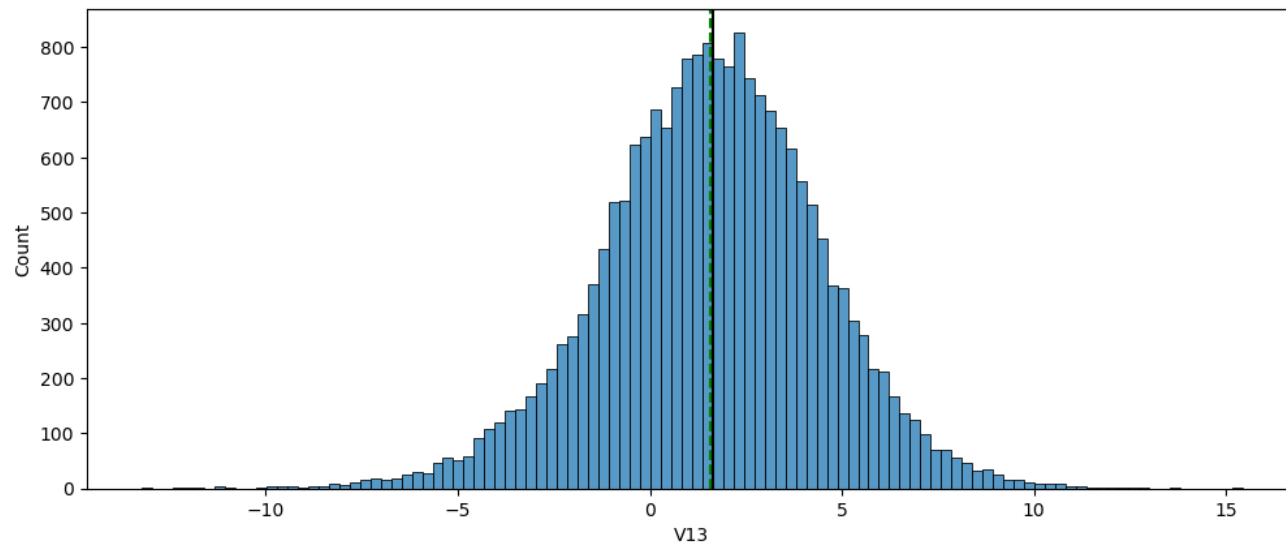
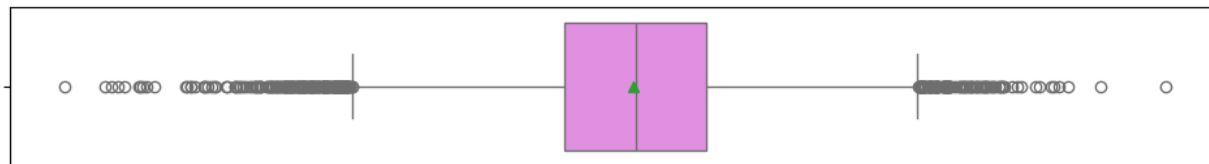
EDA Results: Univariate Analysis Graphs –Variables V1 to V40



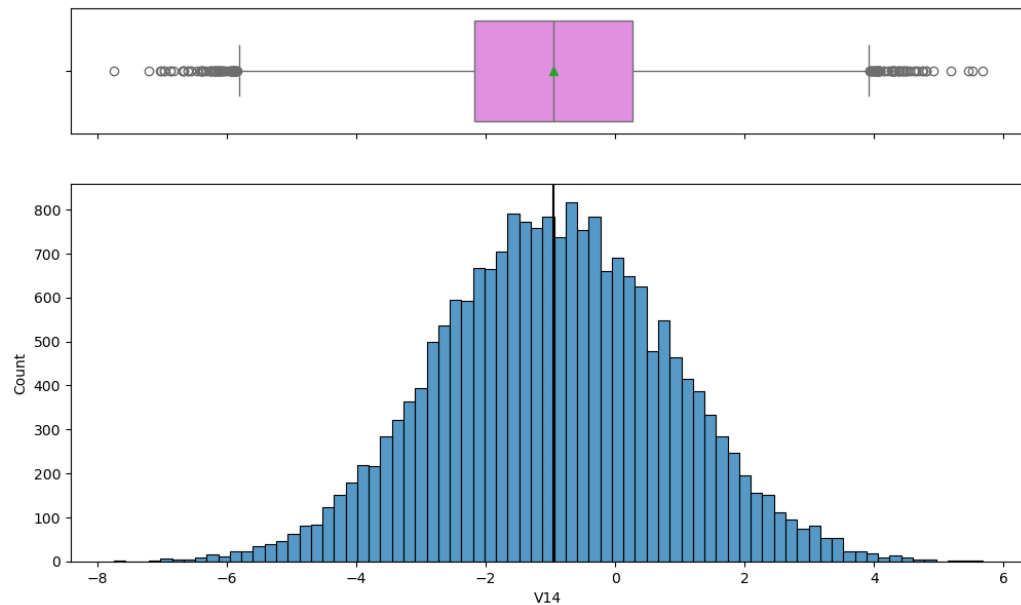
EDA Results: Univariate Analysis Graphs –Variables V1 to V40



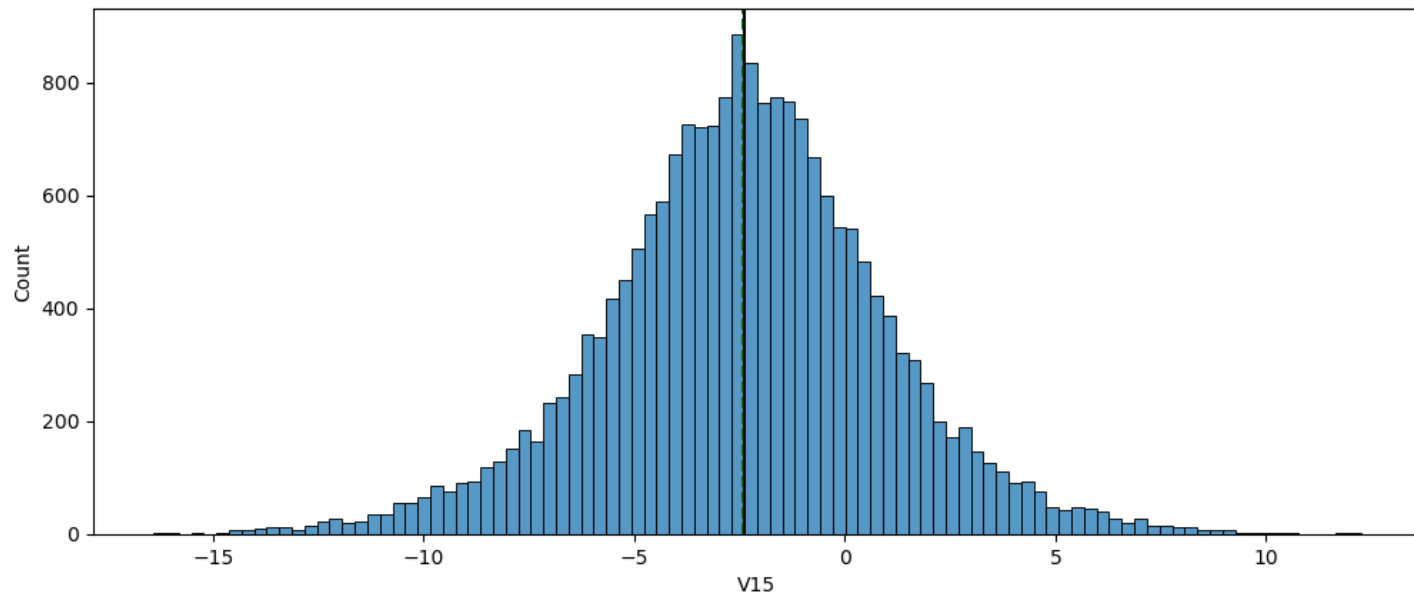
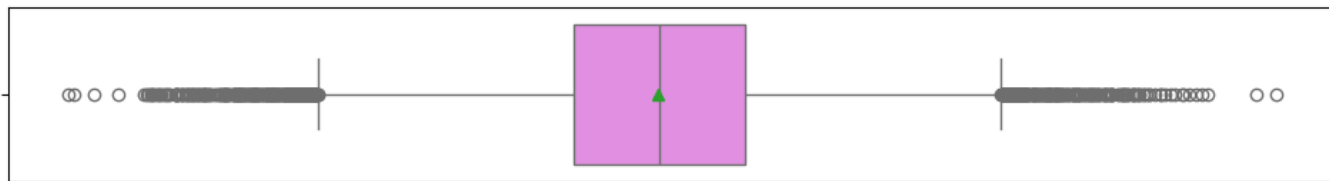
EDA Results: Univariate Analysis Graphs –Variables V1 to V40



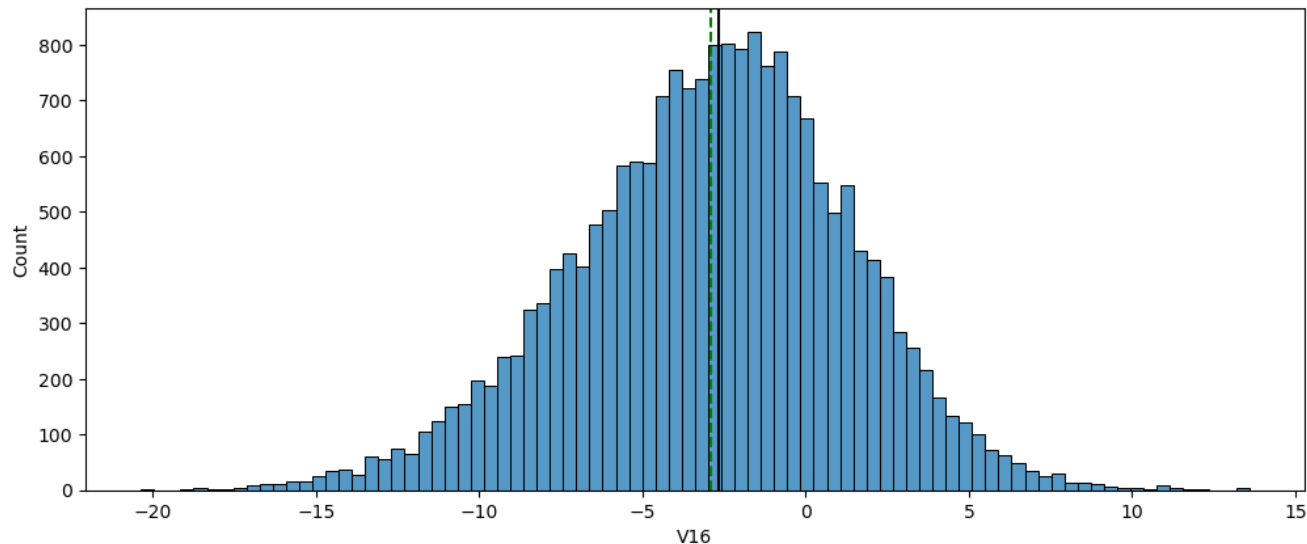
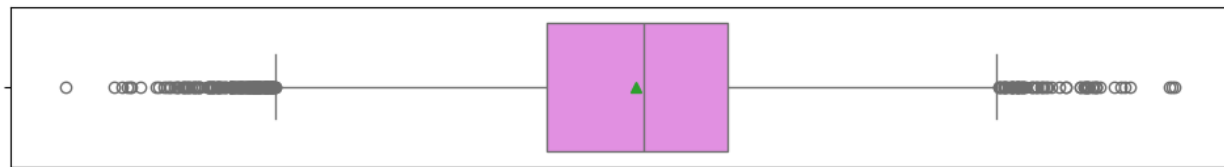
EDA Results: Univariate Analysis Graphs –Variables V1 to V40



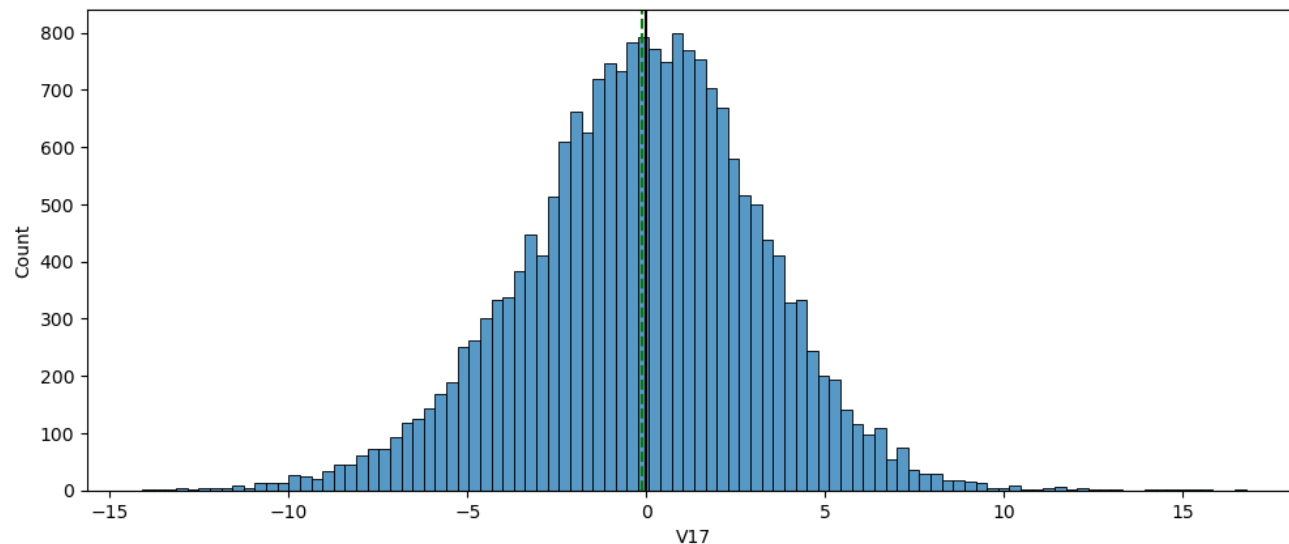
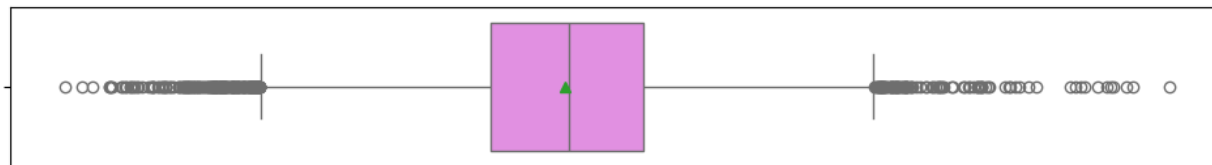
EDA Results: Univariate Analysis Graphs –Variables V1 to V40



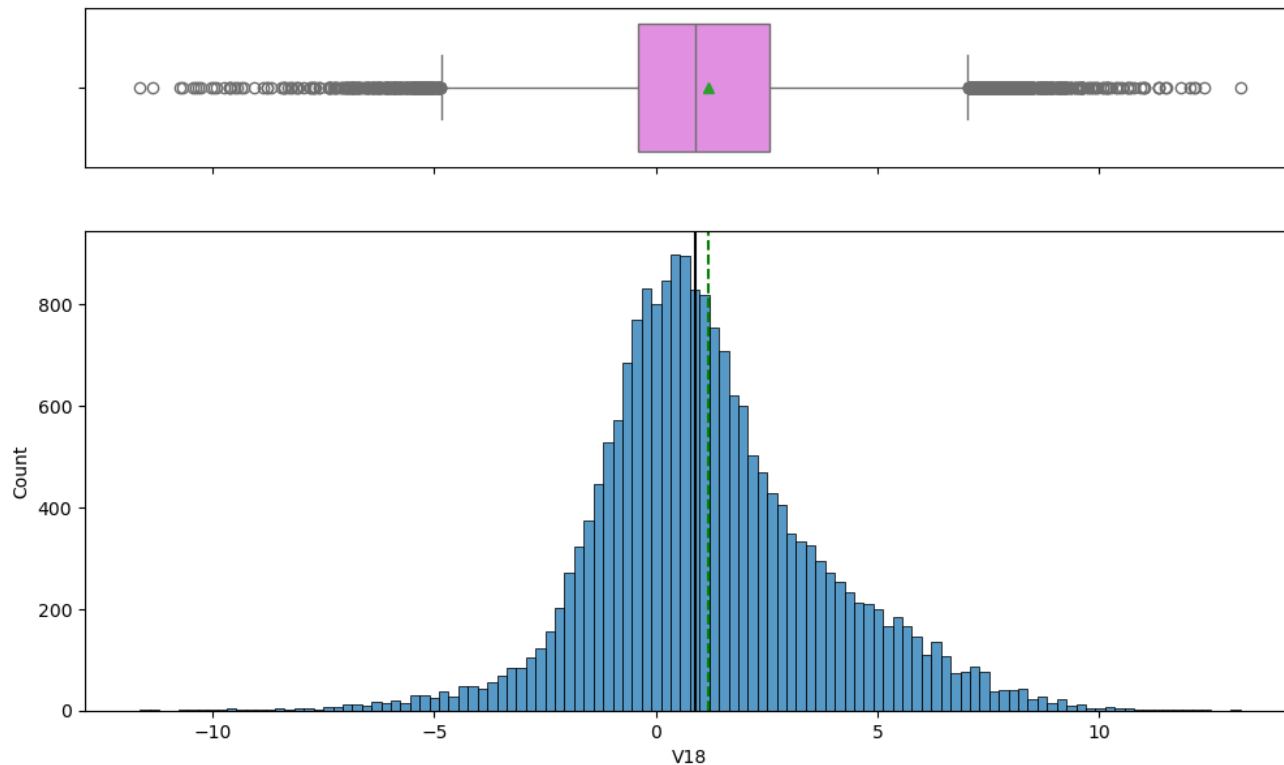
EDA Results: Univariate Analysis Graphs –Variables V1 to V40



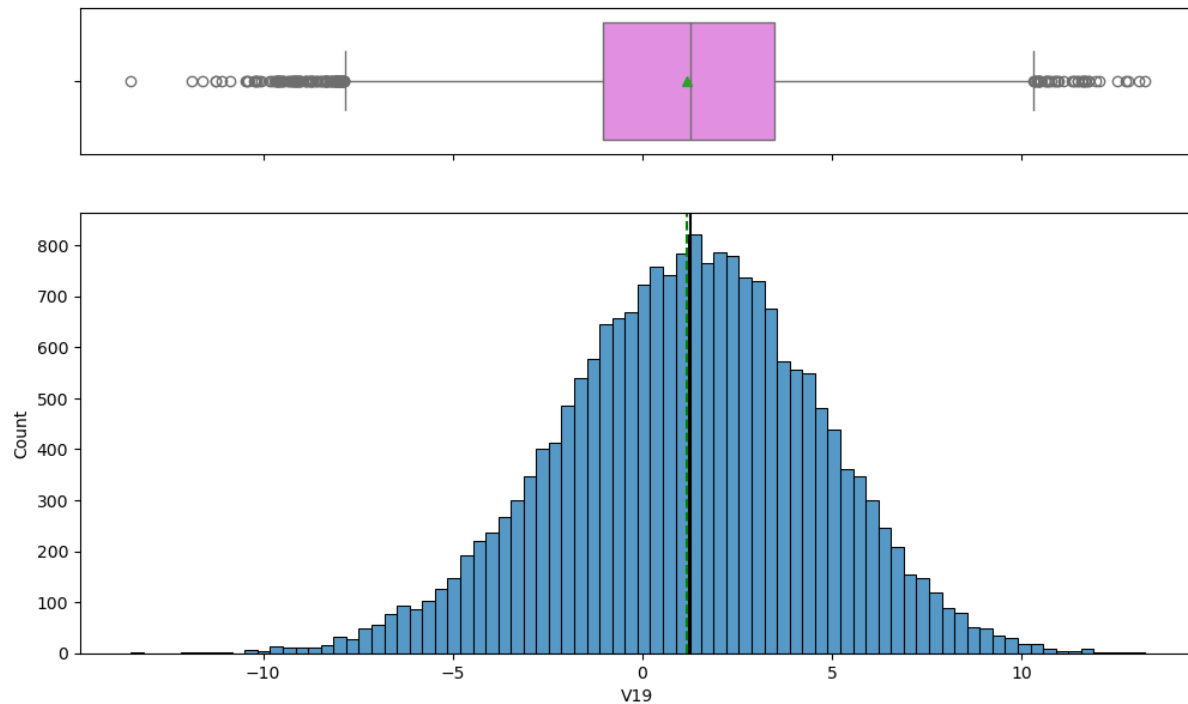
EDA Results: Univariate Analysis Graphs –Variables V1 to V40



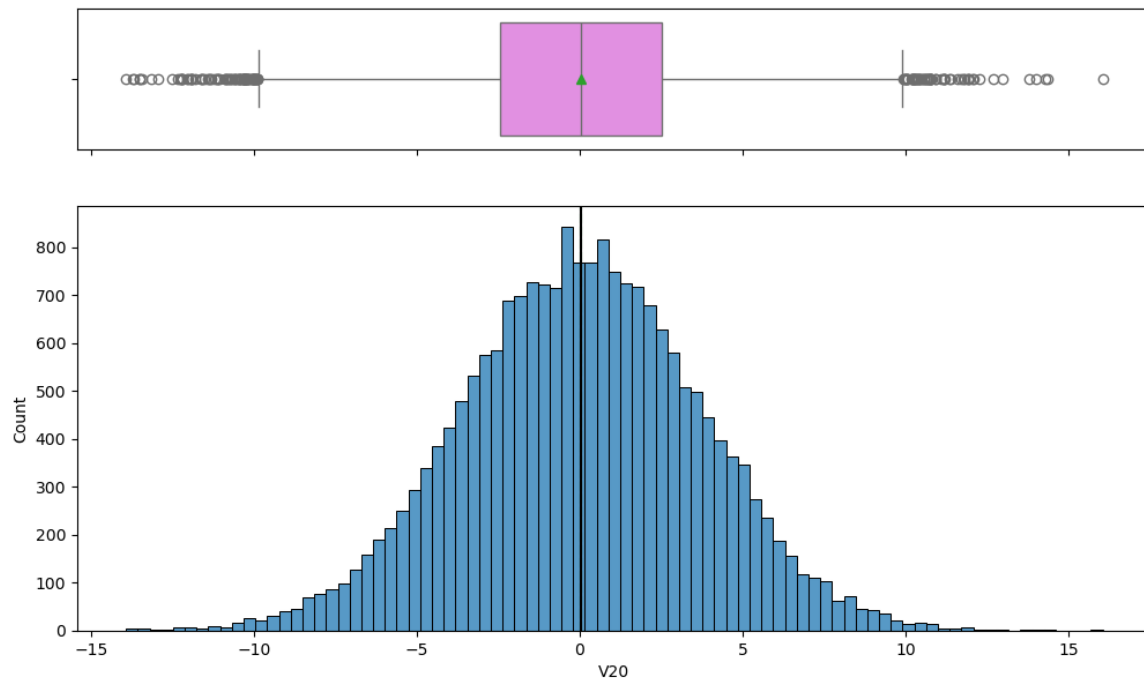
EDA Results: Univariate Analysis Graphs –Variables V1 to V40



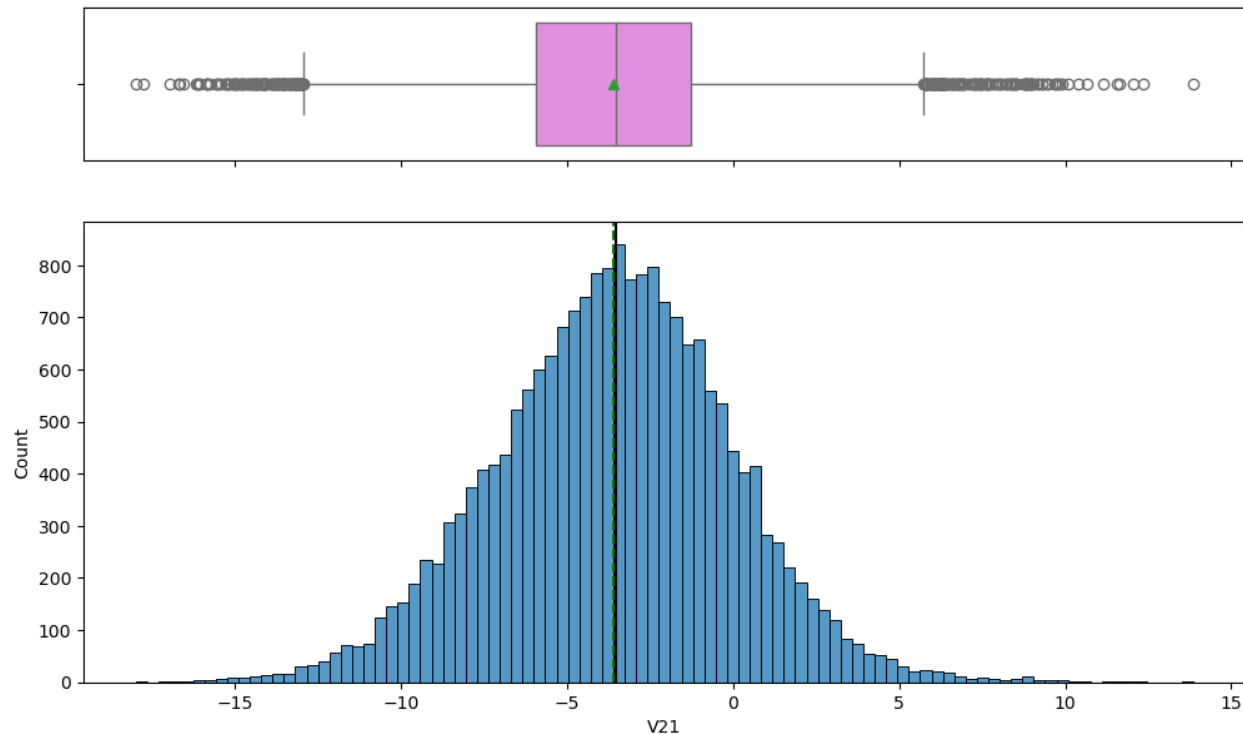
EDA Results: Univariate Analysis Graphs –Variables V1 to V40



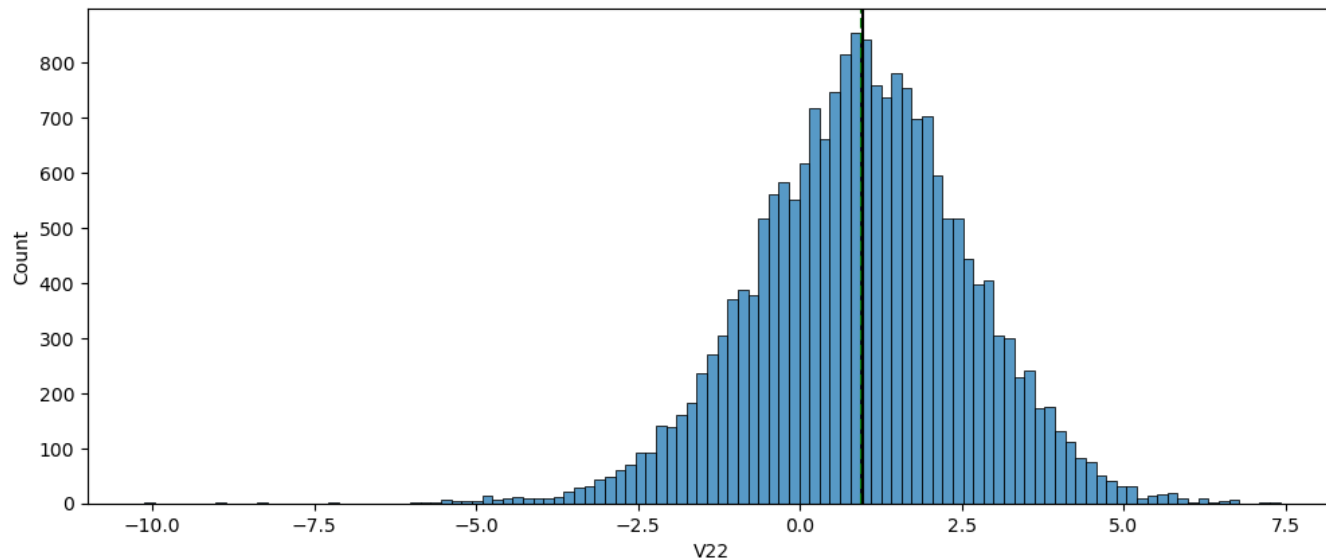
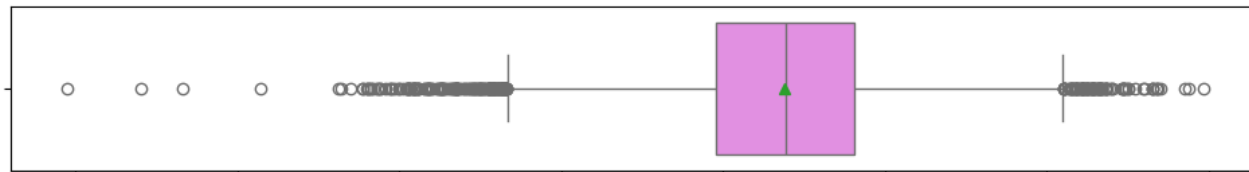
EDA Results: Univariate Analysis Graphs –Variables V1 to V40



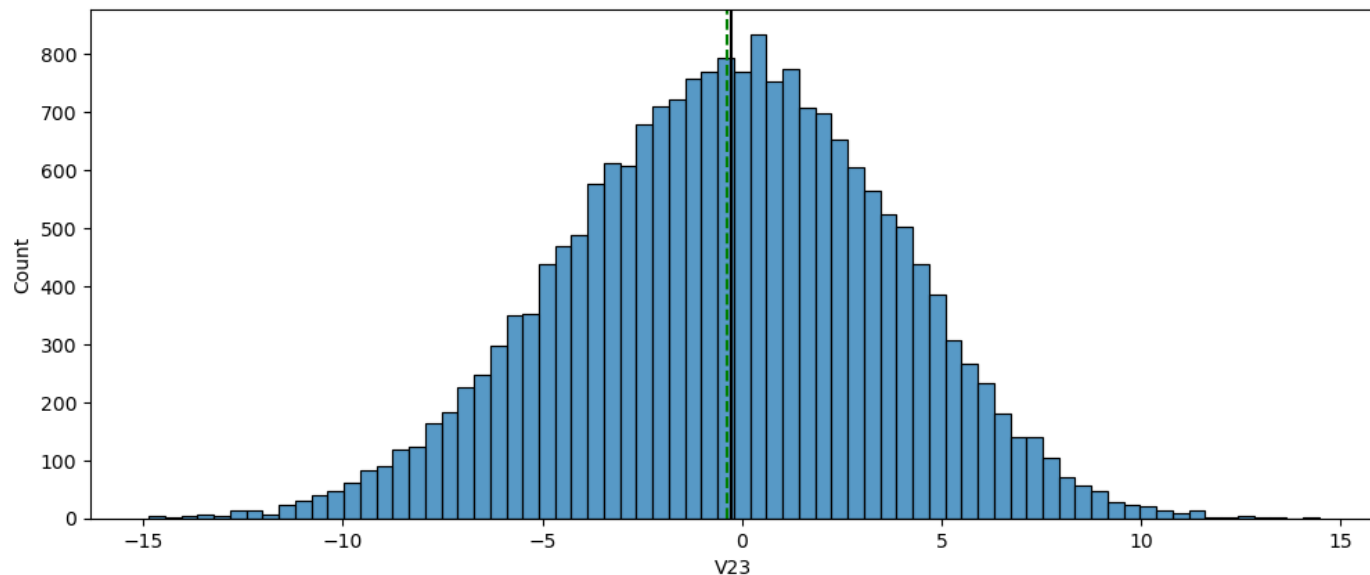
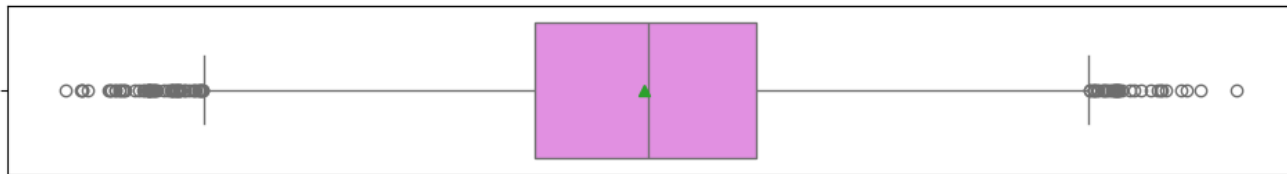
EDA Results: Univariate Analysis Graphs –Variables V1 to V40



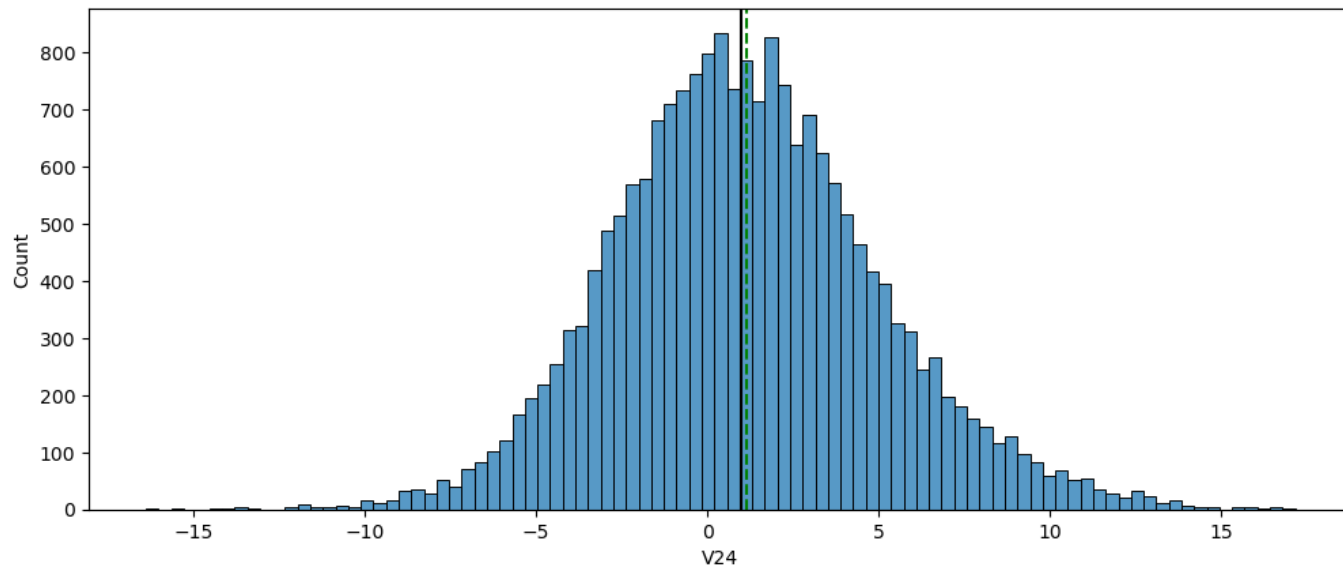
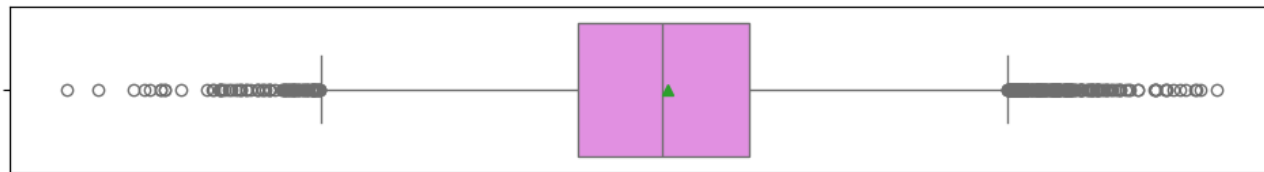
EDA Results: Univariate Analysis Graphs –Variables V1 to V40



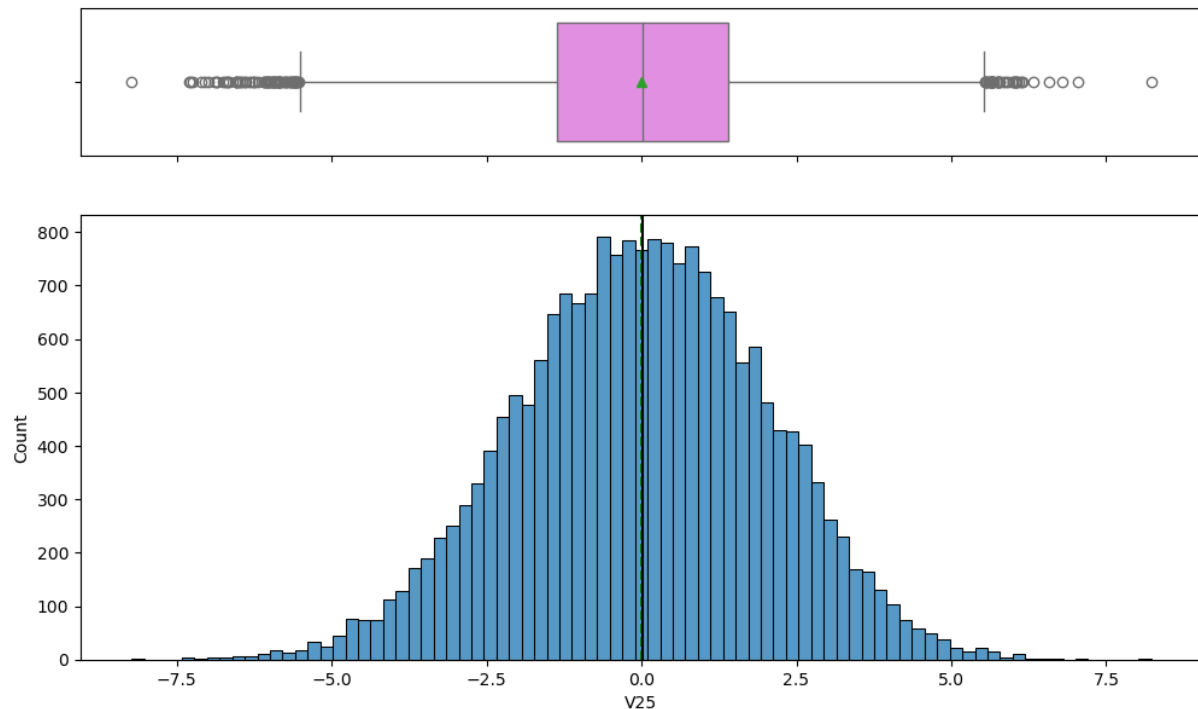
EDA Results: Univariate Analysis Graphs –Variables V1 to V40



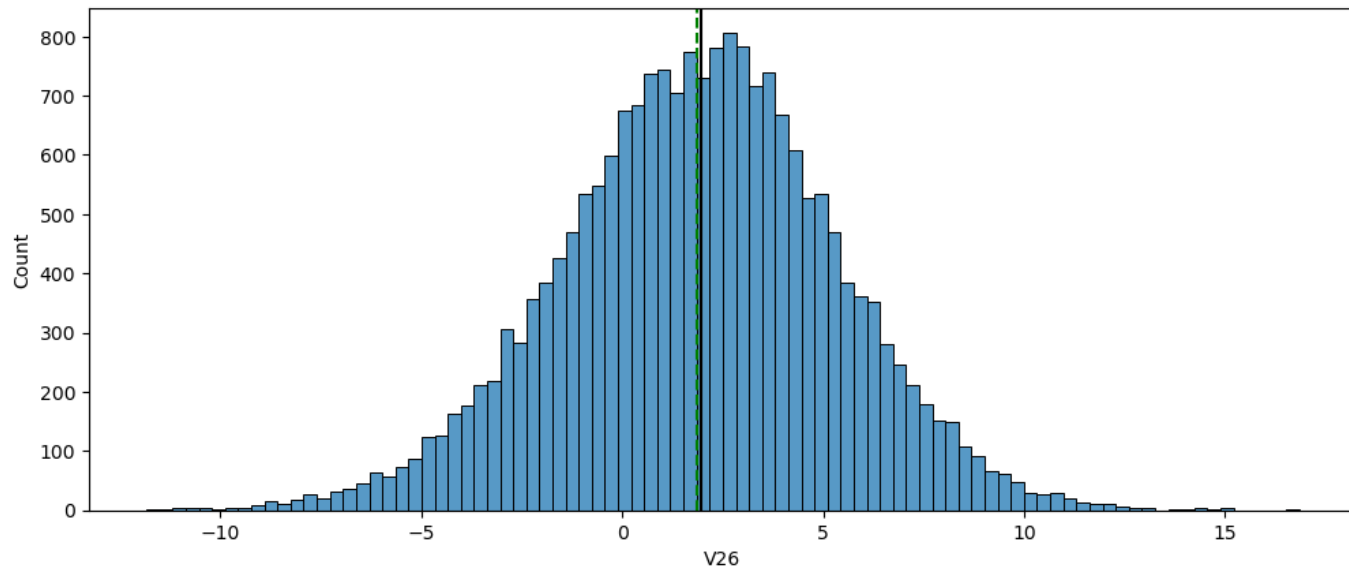
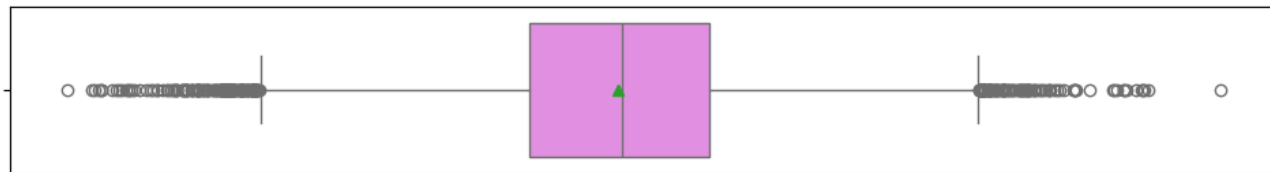
EDA Results: Univariate Analysis Graphs –Variables V1 to V40



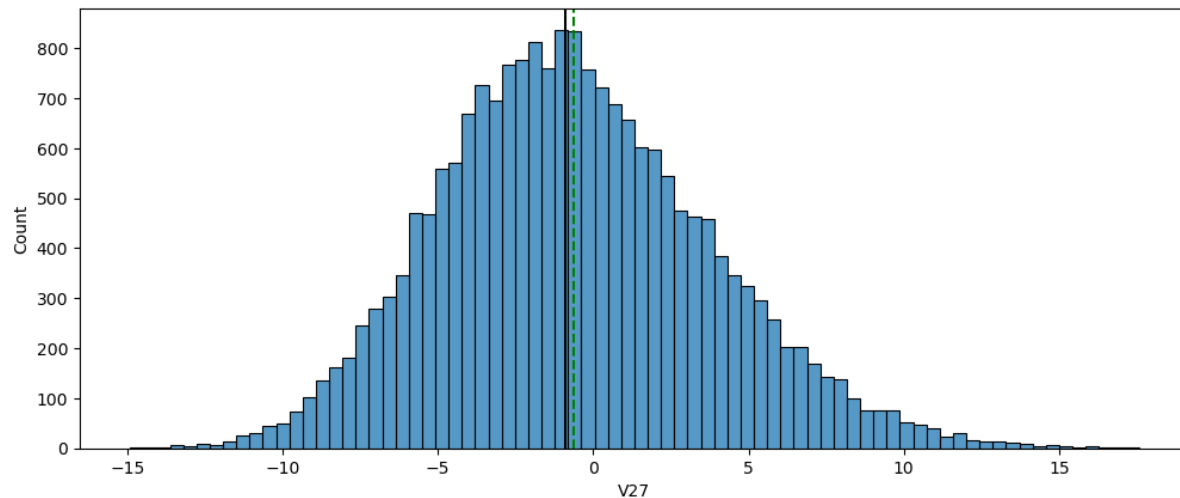
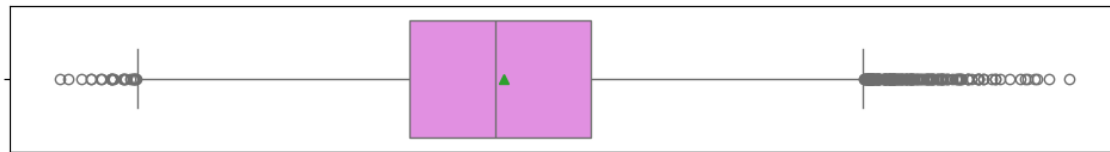
EDA Results: Univariate Analysis Graphs –Variables V1 to V40



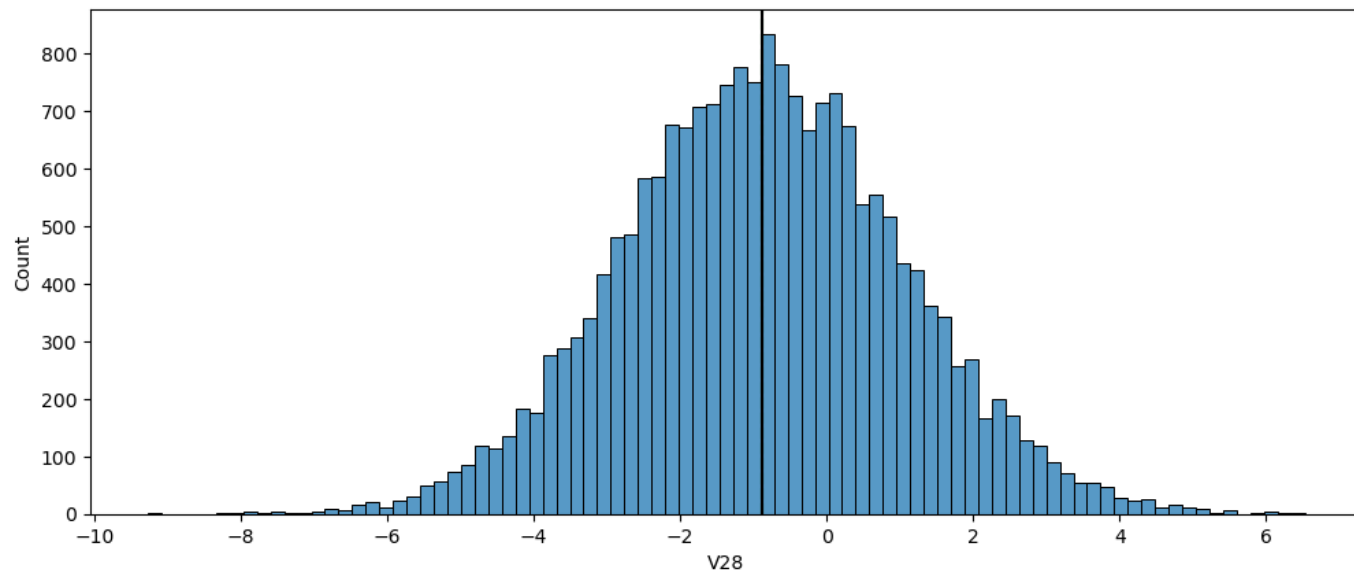
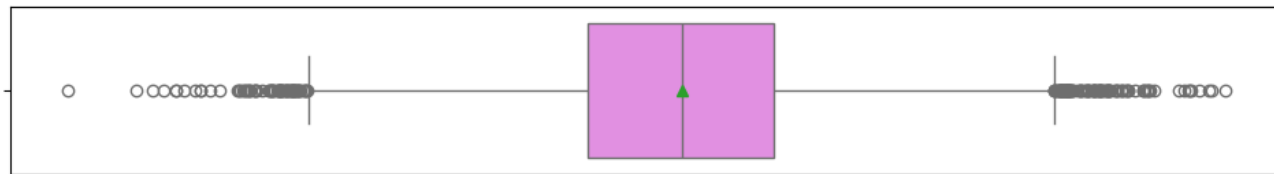
EDA Results: Univariate Analysis Graphs –Variables V1 to V40



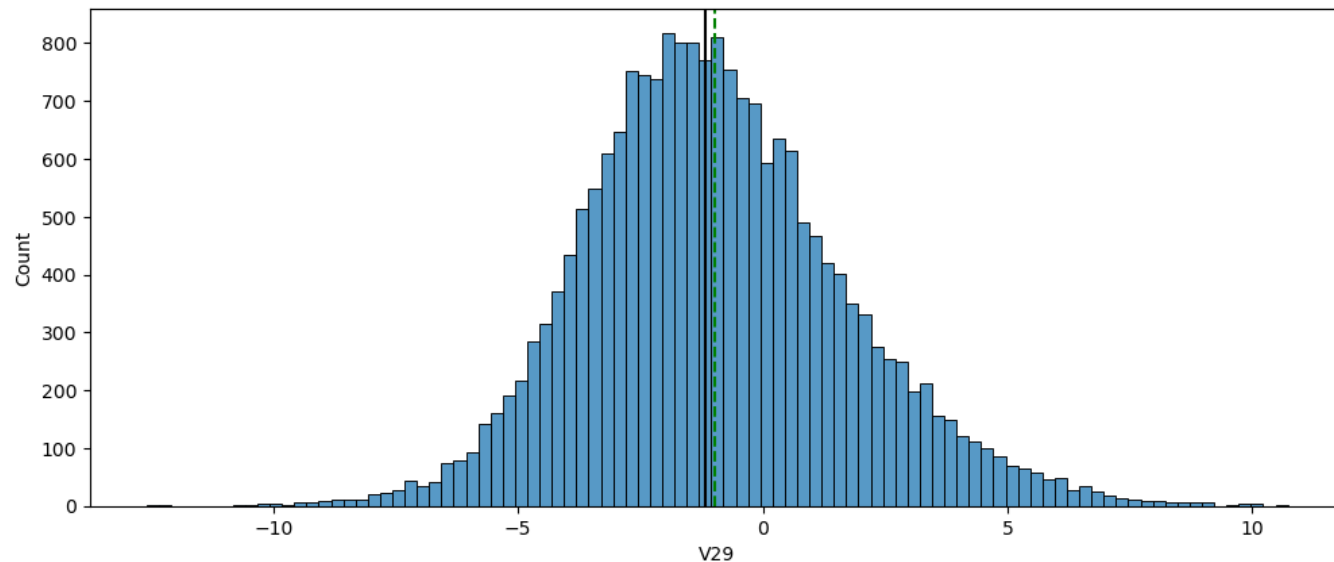
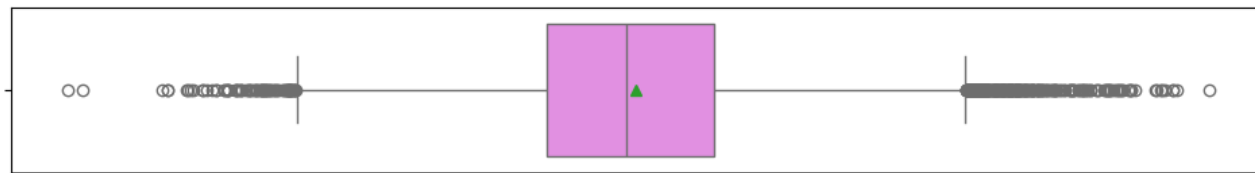
EDA Results: Univariate Analysis Graphs –Variables V1 to V40



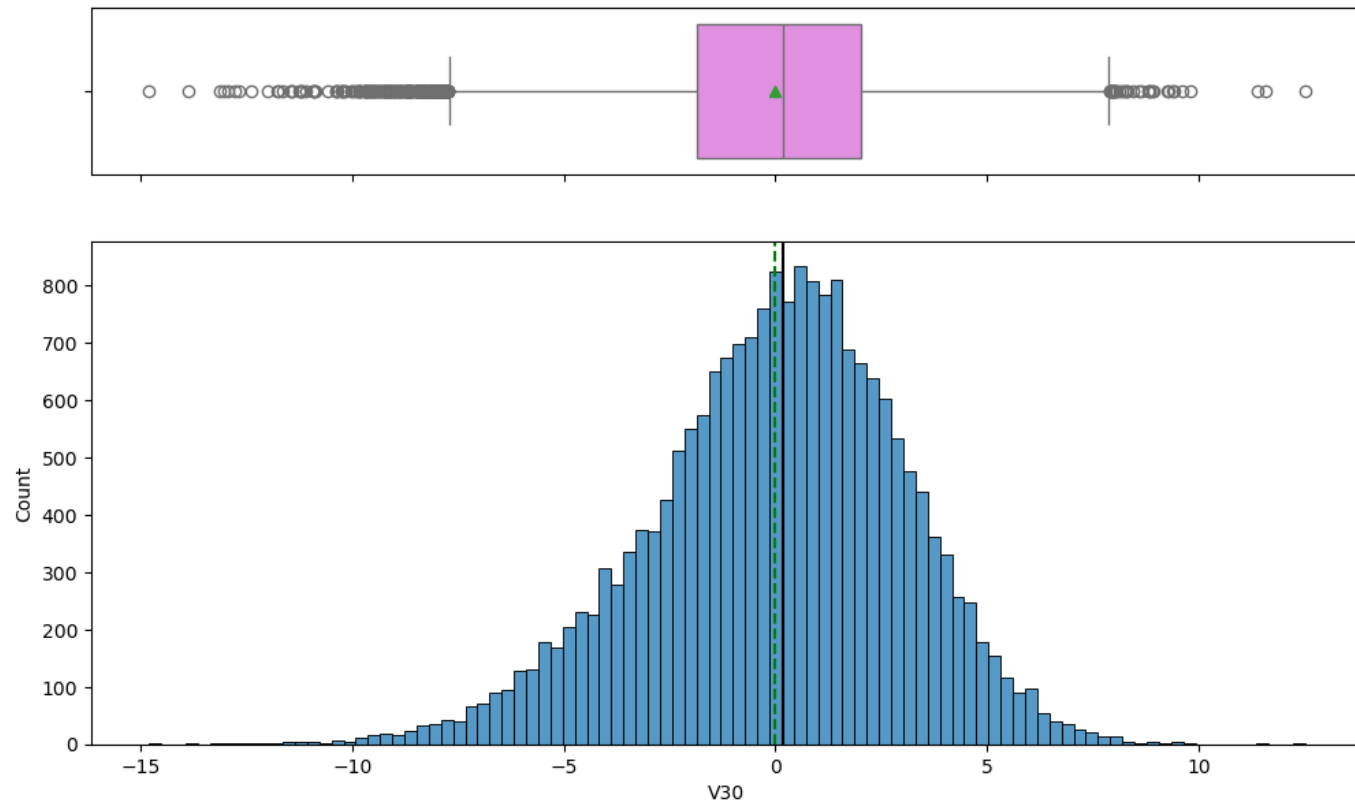
EDA Results: Univariate Analysis Graphs –Variables V1 to V40



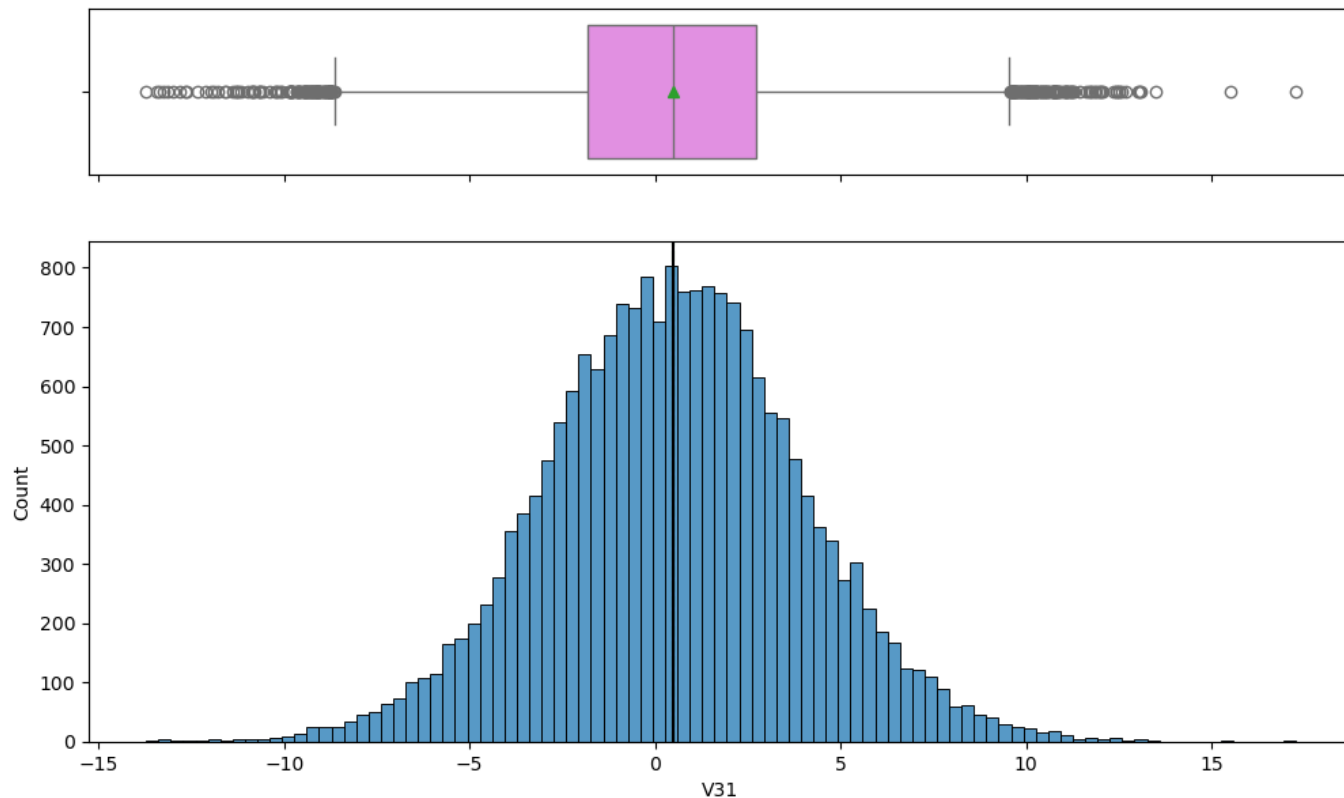
EDA Results: Univariate Analysis Graphs –Variables V1 to V40



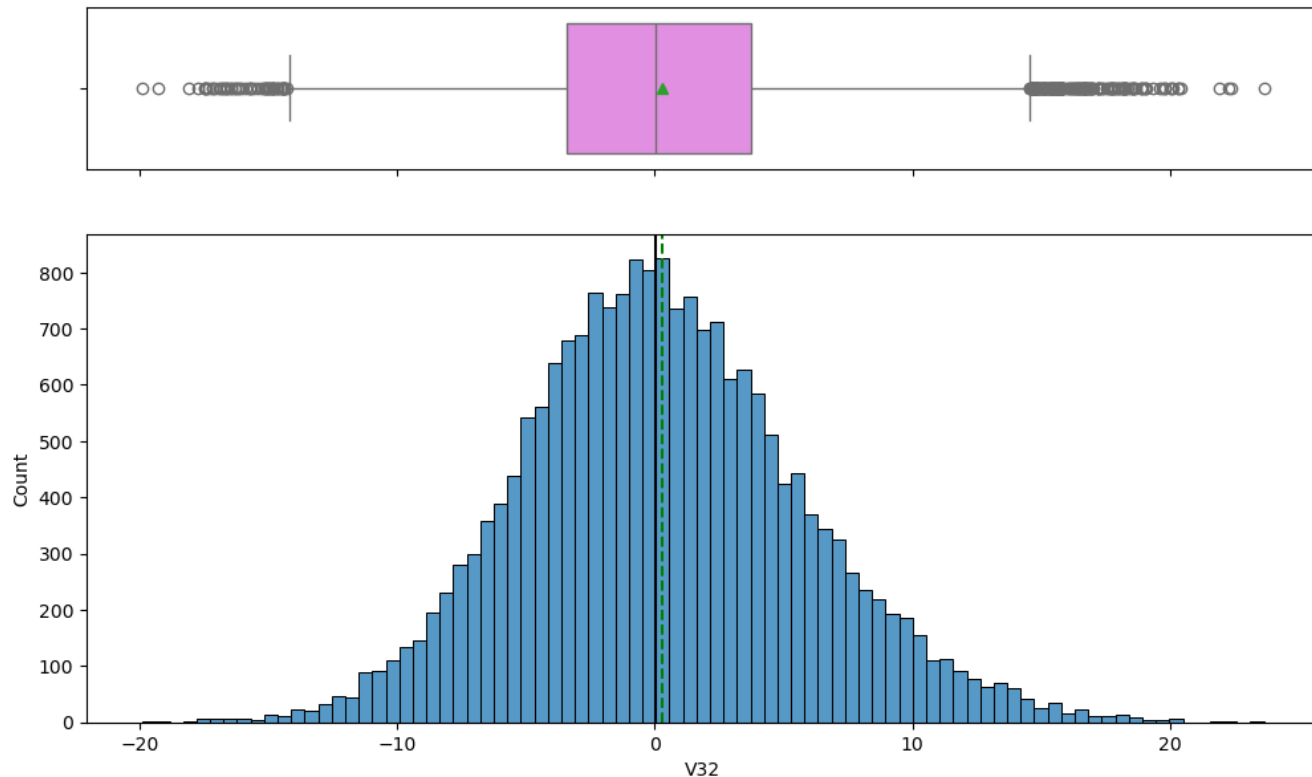
EDA Results: Univariate Analysis Graphs –Variables V1 to V40



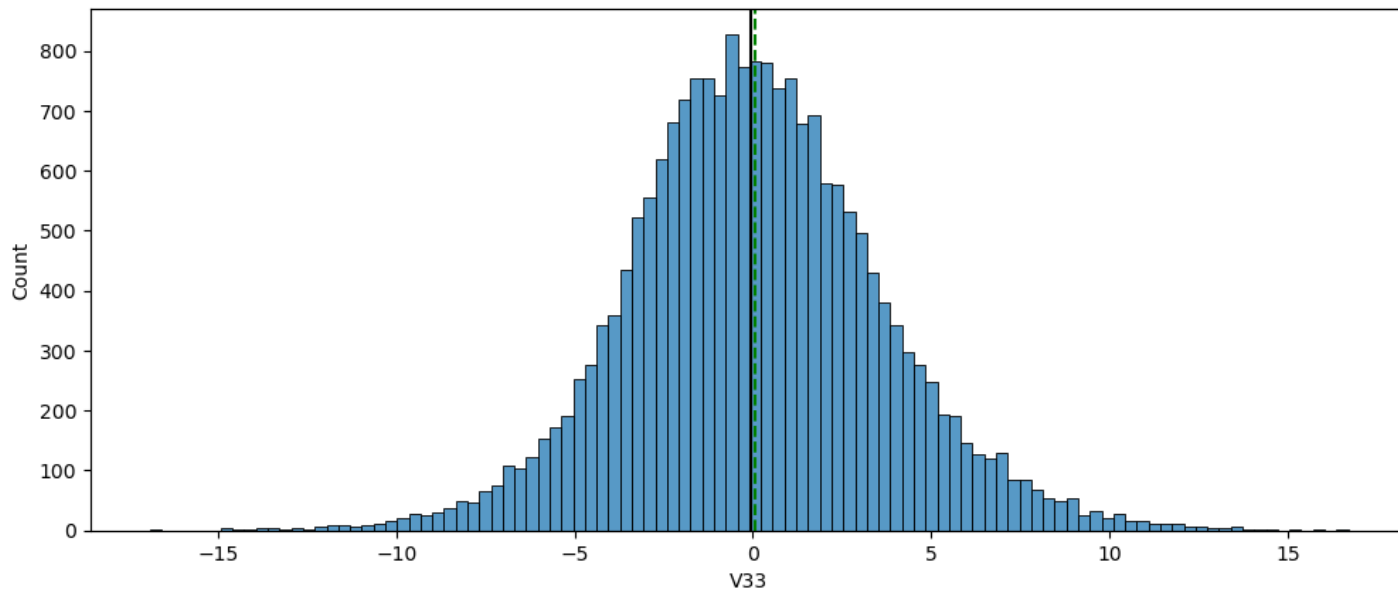
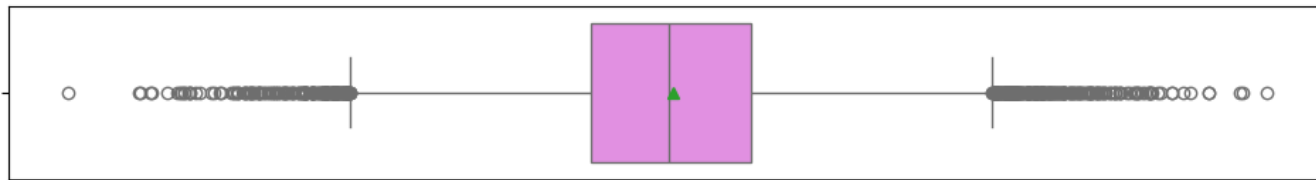
EDA Results: Univariate Analysis Graphs –Variables V1 to V40



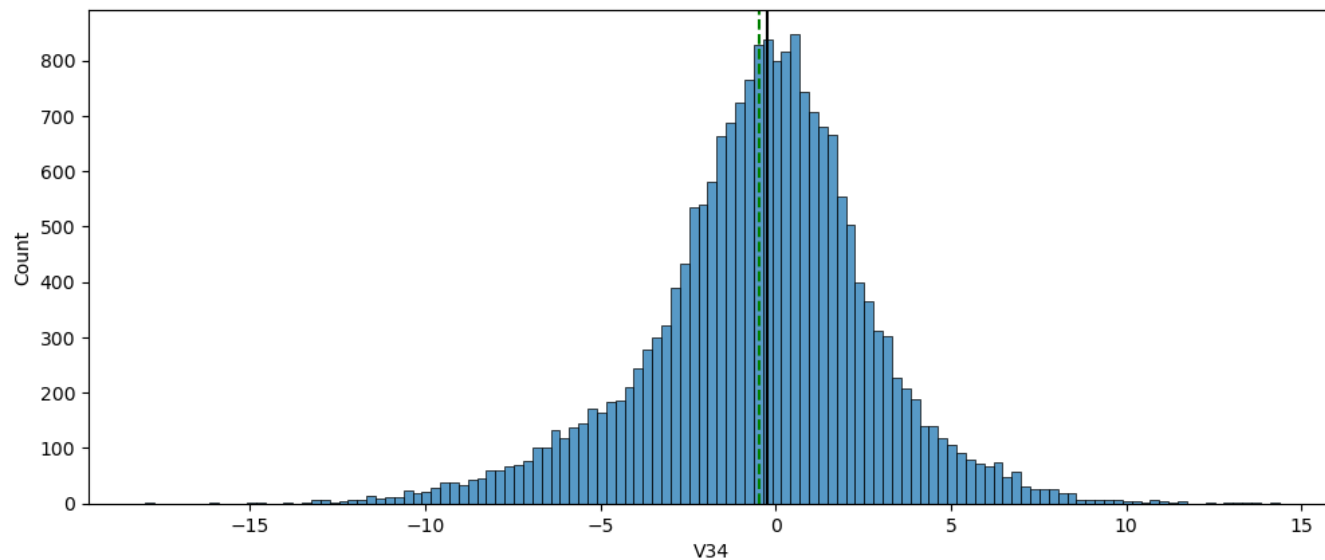
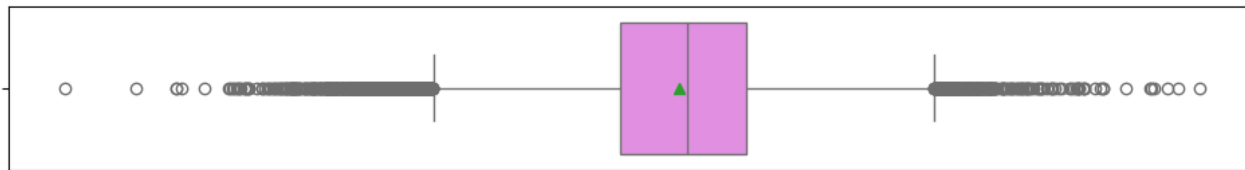
EDA Results: Univariate Analysis Graphs –Variables V1 to V40



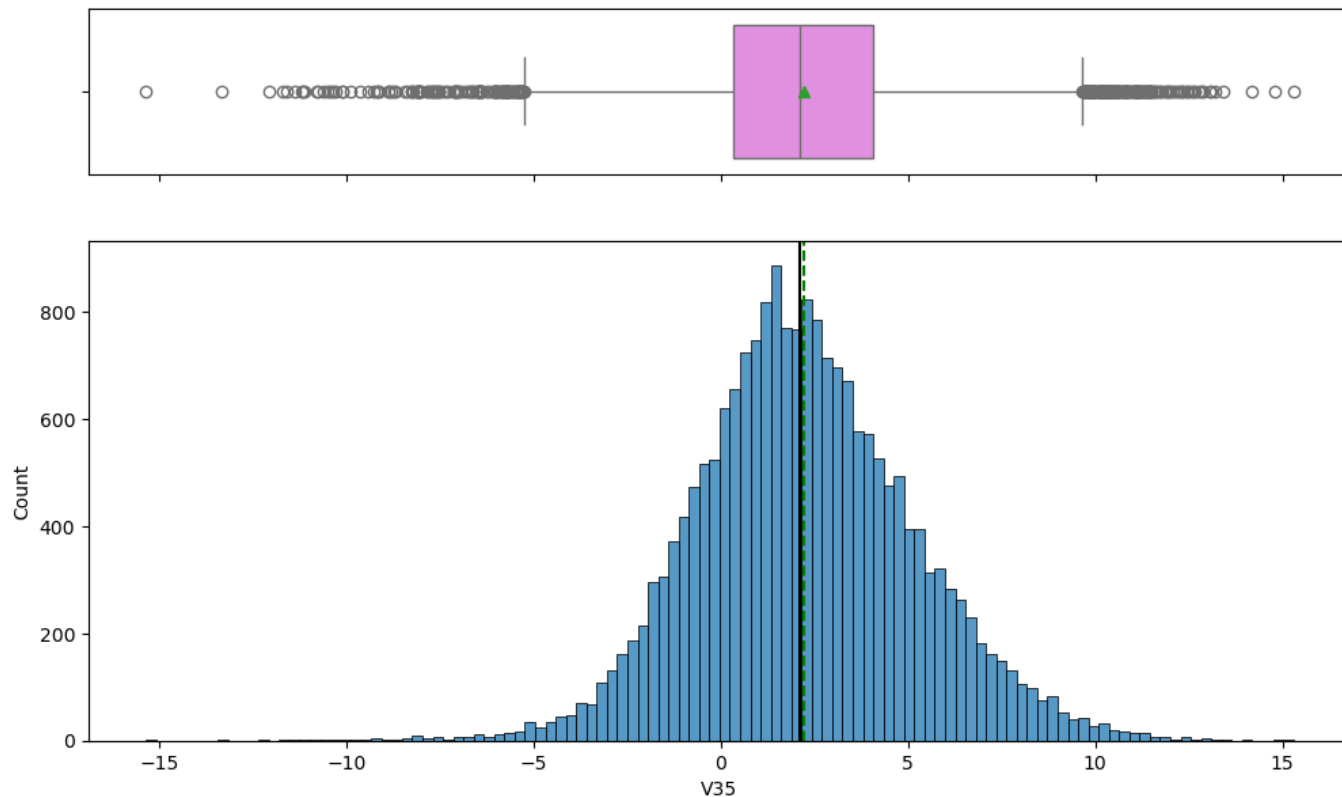
EDA Results: Univariate Analysis Graphs –Variables V1 to V40



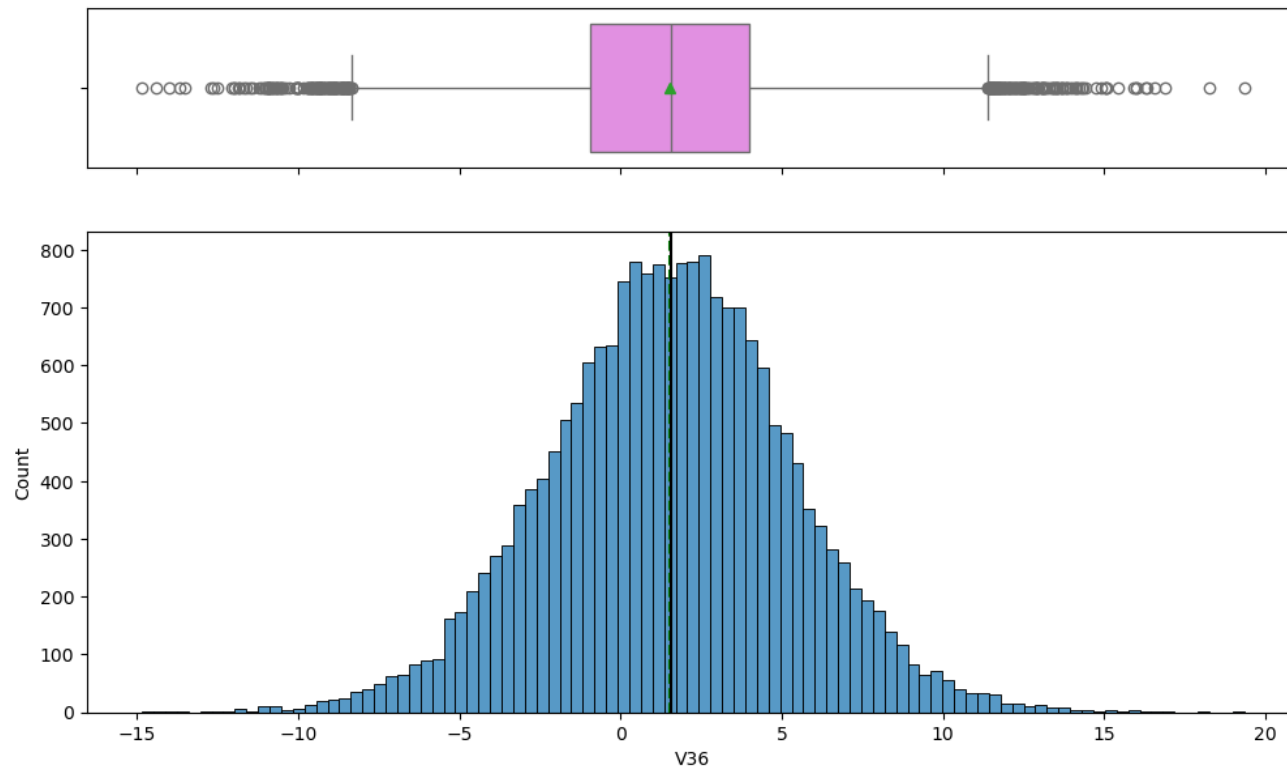
EDA Results: Univariate Analysis Graphs –Variables V1 to V40



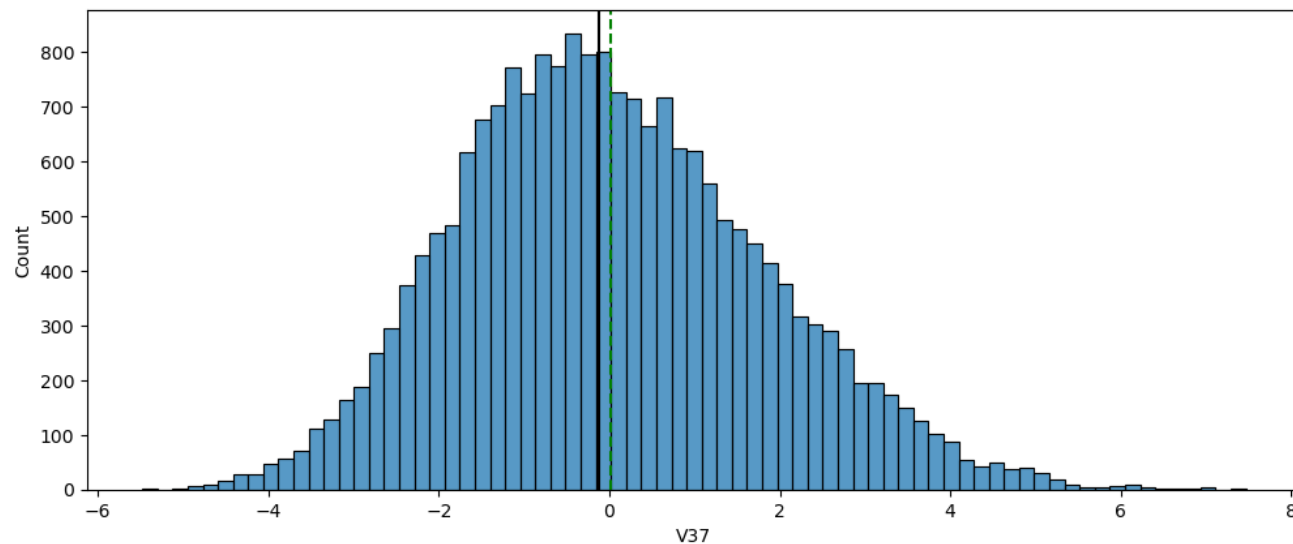
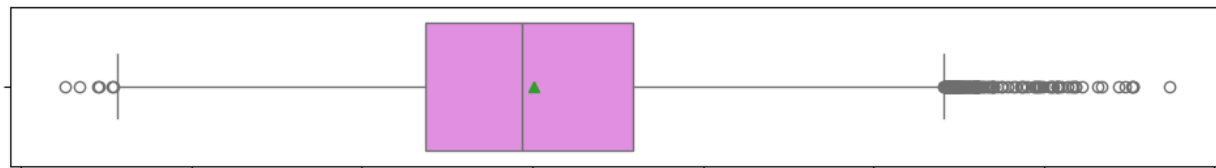
EDA Results: Univariate Analysis Graphs –Variables V1 to V40



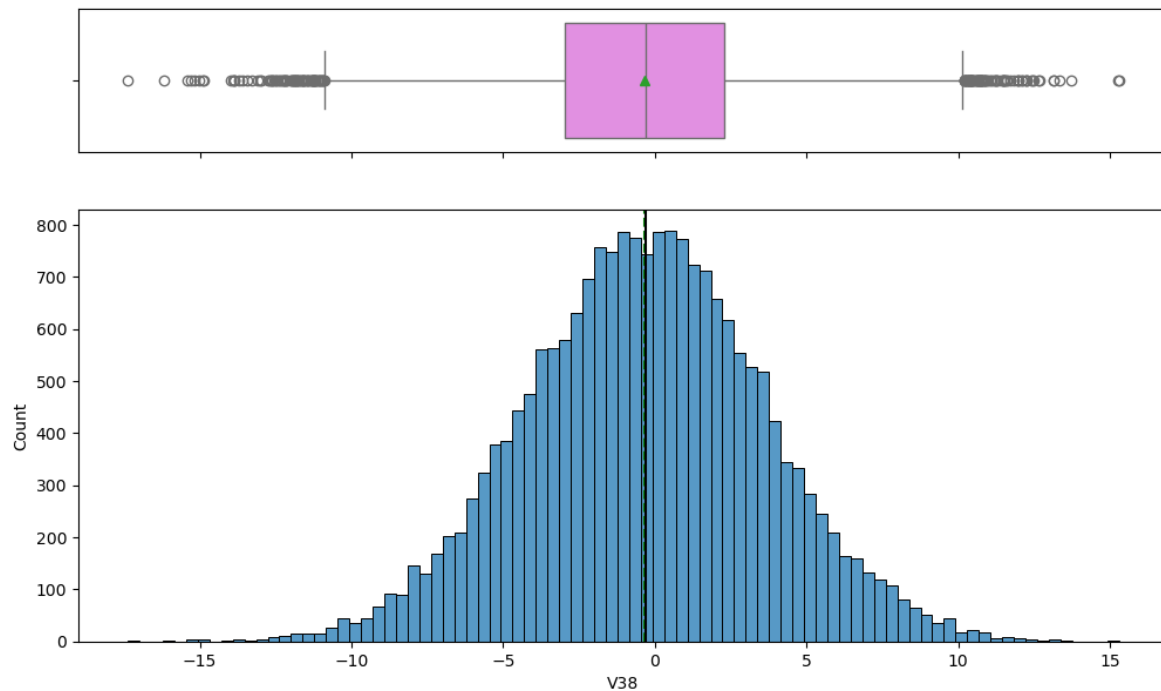
EDA Results: Univariate Analysis Graphs –Variables V1 to V40



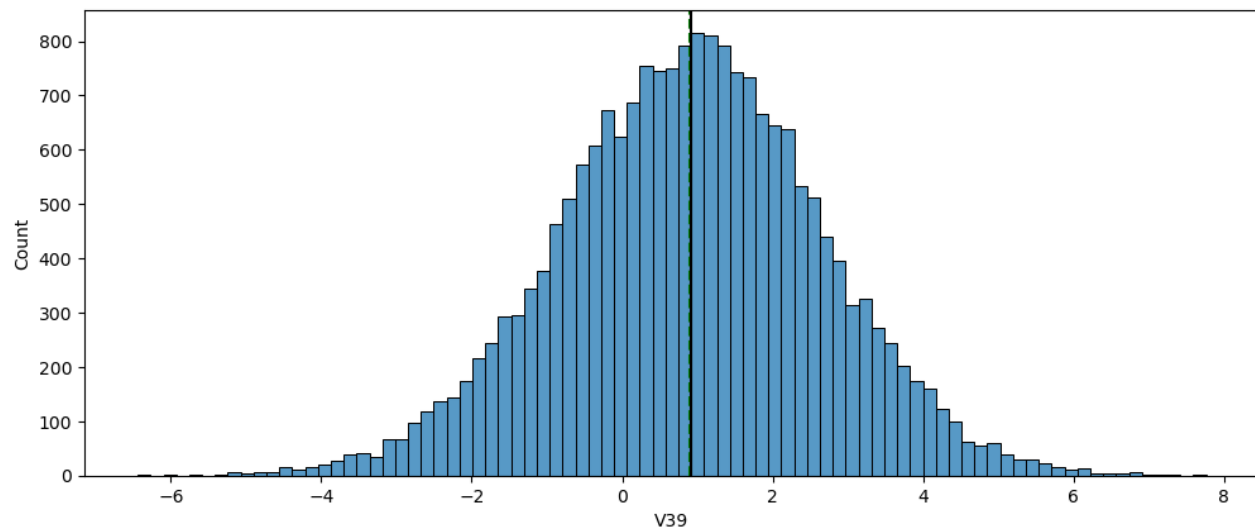
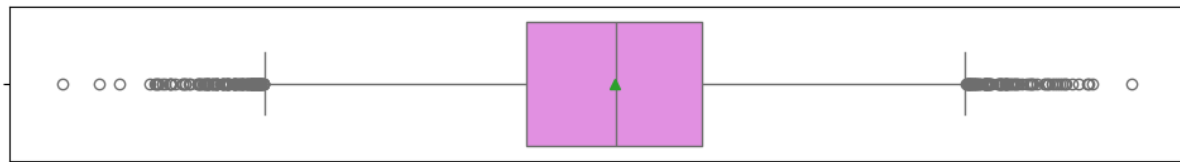
EDA Results: Univariate Analysis Graphs –Variables V1 to V40



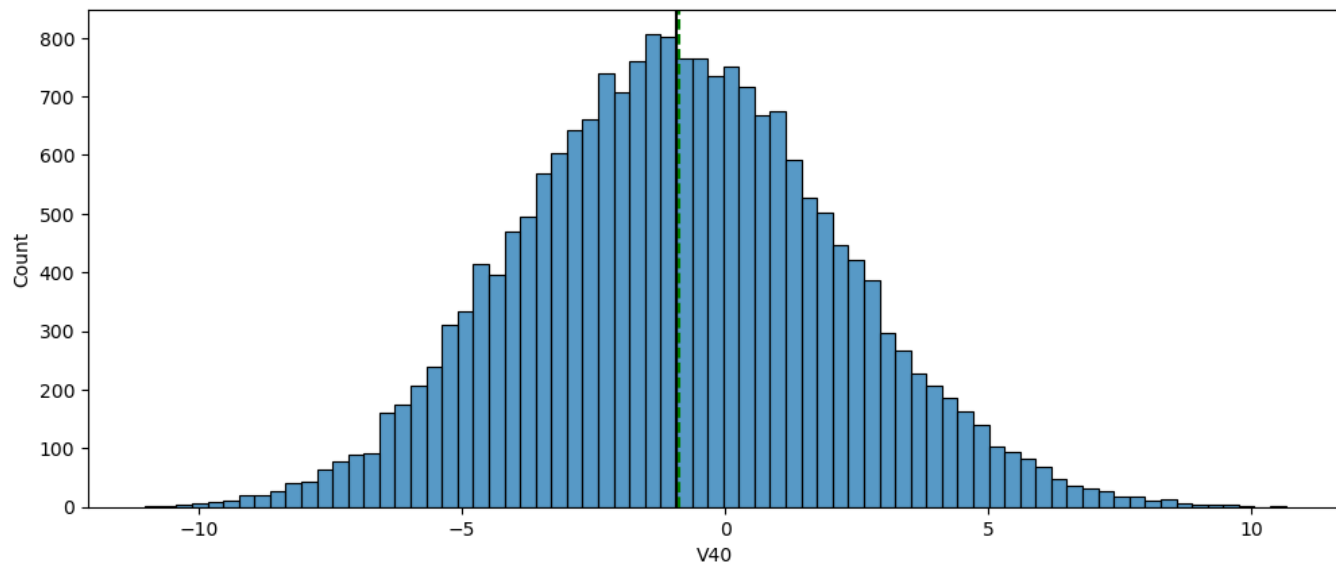
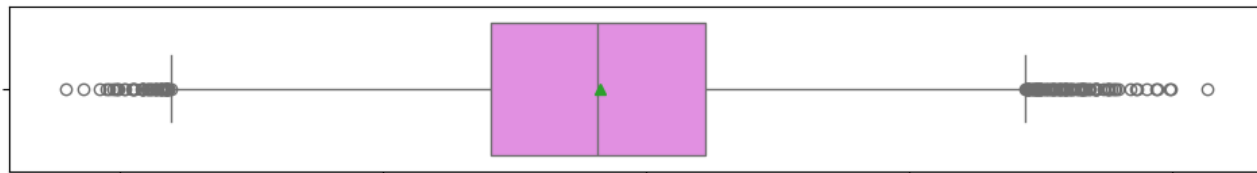
EDA Results: Univariate Analysis Graphs –Variables V1 to V40



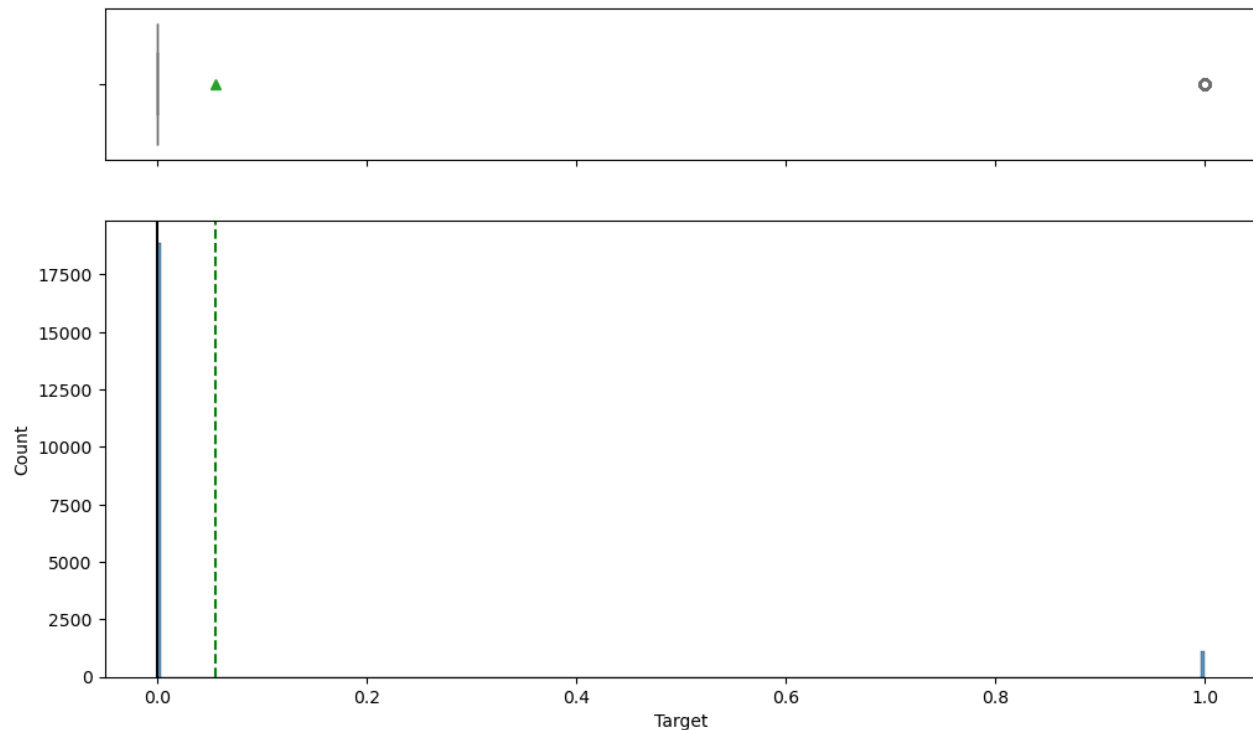
EDA Results: Univariate Analysis Graphs –Variables V1 to V40



EDA Results: Univariate Analysis Graphs –Variables V1 to V40



EDA Results: Univariate Analysis Graphs –Variables V1 to V40





Happy Learning !

